

OPS_OST - Operaciones y tecnología - PoC Big Data Contenedores

 chatgpt.com/g/g-p-6863179af3b48191819689ce39b4ab12/c/6a642b21-14f0-83e9-bc1f-f65f89a949d2

You said:

estoy con tiempo libre y quiero invertirlo en seguir aprendiendo y pensaba en una PoC de bigdata, algo así como replicar datos desde distintos motores de base de datos hacia un cluster de bigdata. tal vez estoy pensando en grande, pero la imagen que te comparto es una arquitectura que me gustaría en su momento, lograr replicar con ayuda de contenedores. tienes preguntas, hazmelas.

ChatGPT said:

La idea es técnicamente viable, pero conviene construirla de manera incremental. La arquitectura de la imagen representa varias capacidades distintas: ingesta batch y en tiempo real, almacenamiento, procesamiento, analítica, visualización, gobierno y seguridad. Intentar implementarlas todas desde el inicio probablemente convertiría la PoC en un proyecto demasiado amplio.

Para definir una arquitectura ejecutable y un roadmap, necesito precisar lo siguiente.

1. Objetivo principal

1. ¿Qué quieres demostrar con la PoC?
 - Replicación entre motores heterogéneos.
 - Captura de cambios en tiempo real mediante CDC.
 - Construcción de un data lake.
 - Procesamiento distribuido.
 - Analítica y visualización.
 - Comparación entre procesamiento batch y streaming.
 - Una combinación de los anteriores.
2. ¿El foco será principalmente académico, técnico-operacional o demostrativo para un entorno empresarial?

3. ¿Quieres simular un caso real, por ejemplo:

- operaciones financieras;
- transacciones comerciales;
- sensores o telemetría;
- inventario de infraestructura;
- datos geoespaciales;
- datos relacionados con tu tesis?

2. Fuentes de datos

4. ¿Qué motores te interesa integrar inicialmente?

- PostgreSQL.
- MySQL/MariaDB.
- Oracle Database.
- SQL Server.
- MongoDB.
- Archivos CSV, JSON o Parquet.
- APIs.

5. ¿Los motores se levantarán como contenedores de laboratorio o pretendes conectarte a bases ya existentes?

6. ¿Qué modalidad quieres probar?

- Carga completa inicial.
- Cargas incrementales por fecha o identificador.
- CDC mediante lectura de logs transaccionales.
- Batch programado.
- Streaming continuo.

7. ¿Cuánto volumen quieres simular?

- Miles de registros.
- Millones de registros.
- Decenas de millones.
- Un flujo continuo de eventos.

Para una primera PoC, sugeriría entre uno y cinco millones de registros, suficiente para observar diferencias sin sobrecargar innecesariamente tu laptop.

3. Plataforma disponible

8. ¿La PoC se ejecutará en tu ThinkPad Windows 11 con 32 GB de RAM y Podman?

9. ¿Cuánto espacio en disco puedes asignarle al laboratorio?

10. ¿Podman está operando mediante `podman machine` con WSL2 o mediante otra virtualización?
11. ¿Quieres que toda la solución sea compatible con `podman compose`?
12. ¿La solución debe funcionar completamente sin servicios cloud?
13. ¿Te interesa que posteriormente pueda migrarse a servidores Linux o Kubernetes?

4. Tecnologías que quieres aprender

14. ¿Qué tecnologías tienes mayor interés en incorporar?

Capa	Alternativas
Ingesta batch	Apache NiFi, Airbyte, Kafka Connect, scripts Python
CDC	Debezium, Kafka Connect
Mensajería	Apache Kafka, Redpanda
Data lake	MinIO, almacenamiento local, HDFS
Formato de datos	Parquet, Avro, JSON
Procesamiento	Apache Spark, Flink, Python
Consultas SQL	Trino, Spark SQL, DuckDB
Orquestación	Airflow, Dagster
Catálogo	DataHub, OpenMetadata
Calidad	Great Expectations, Soda
Visualización	Superset, Metabase, Grafana
Observabilidad	Prometheus, Grafana, Loki

15. ¿Quieres aprender tecnologías clásicas del ecosistema Hadoop —HDFS, Hive, YARN— o prefieres una arquitectura moderna basada en almacenamiento de objetos, Parquet, Kafka, Spark y Trino?

Esta decisión es importante. Para una PoC actual y ejecutable en una laptop, recomendaría una arquitectura moderna sin comenzar por un clúster Hadoop tradicional.

5. Arquitectura batch y streaming

16. ¿Quieres implementar desde el principio las dos rutas?
- **Batch:** base de datos → extracción → archivos Parquet → data lake → Spark/Trino.
 - **Streaming:** base de datos → CDC → Kafka → procesamiento → almacenamiento analítico.
17. ¿Qué latencia sería aceptable para la ruta de tiempo real?
- Segundos.
 - Menos de un minuto.
 - Entre uno y cinco minutos.
 - Solo una demostración conceptual.
18. ¿Quieres conservar el historial completo de cambios, incluyendo inserciones, actualizaciones y eliminaciones?

6. Resultado esperado

19. ¿Qué salida te gustaría tener al finalizar la primera versión?
- Dashboard.
 - Consultas SQL federadas.
 - Data lake consultable.
 - Comparación entre origen y destino.
 - Métricas de latencia y throughput.
 - Trazabilidad de cada registro.
 - Recuperación después de una caída.
 - Todas las anteriores, por etapas.
20. ¿Quieres que la PoC incluya pruebas operacionales, por ejemplo:
- caída de Kafka;
 - reinicio de un conector;
 - pérdida temporal de conectividad;
 - reprocesamiento;
 - detección de duplicados;
 - validación de consistencia;
 - recuperación desde checkpoints?

7. Alcance de gobierno y seguridad

21. ¿Deseas incorporar desde el comienzo alguno de estos elementos?

- Gestión centralizada de secretos.
- TLS entre componentes.
- Control de acceso.
- Enmascaramiento de datos.
- Catálogo de metadatos.
- Linaje de datos.
- Auditoría.
- Reglas de calidad.

22. ¿La PoC utilizará únicamente datos sintéticos o anonimizados?

Para un laboratorio personal recomiendo datos sintéticos, evitando credenciales, esquemas o datos reales de la organización.

8. Nivel de automatización

23. ¿Quieres que el laboratorio se despliegue con un único comando, por ejemplo:

```
podman compose up -d
```

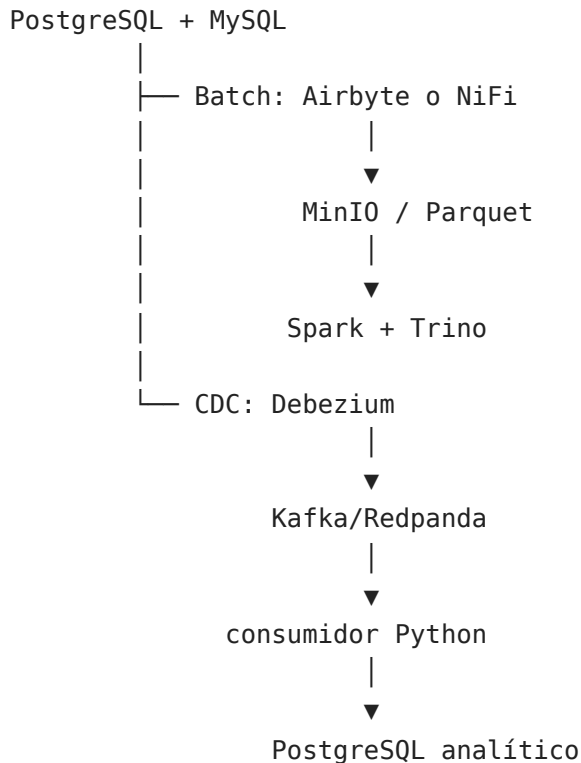
24. ¿Esperas que el proyecto incluya:

- archivos `compose.yml`;
- archivos `.env.example`;
- scripts de inicialización;
- generador de datos;
- pruebas de humo;
- health checks;
- dashboards;
- documentación;
- scripts de respaldo y limpieza?

25. ¿Quieres mantener versionadas las distintas etapas, por ejemplo `v0.1`, `v0.2` y `v1.0`?

Propuesta inicial recomendada

Con tus antecedentes en Podman, Python, Oracle, Linux y operaciones, comenzaría con esta arquitectura:



Visualización: Apache Superset

Observabilidad: Prometheus + Grafana

La implementaríamos en cuatro etapas:

1. **MVP batch:** dos bases de datos, extracción, MinIO, Parquet y consultas.
2. **CDC:** Debezium y Kafka para replicación incremental.
3. **Procesamiento:** Spark para transformaciones y consolidación.
4. **Operabilidad:** métricas, calidad, reinicios, idempotencia y recuperación.

Para comenzar, necesito que definas estas cinco decisiones: **caso de uso, motores de origen, batch/CDC, tecnologías prioritarias y recursos máximos que asignarás a Podman**. Con eso se puede diseñar una primera arquitectura concreta sin sobredimensionarla.

You said:

respondo un par de cosas para ver si en 1 hora invertida podemos lograr algo interesante:

1. pienso en una prueba de concepto de replicación de datos (ingesta y replicación en un almacenamiento).
2. el origen serían 3 o 4 motores de bases de datos como mysql, postgresql, oracle.
3. cada motor de base de datos, debería tener una base de datos de empleos, como oracle tiene el esquema hr, mysql debe tener el suyo y así con el resto. O bien, crear un modelo pequeño de 5 tablas de datos y poblarlas antes de replicar.

4. la replicación sería directamente a un cluster hadoop, pensando en 1 maestro y 2 esclavos.
5. creo que esto sería ingesta de datos y almacenamiento. Pensando en una arquitectura de 5 capas: ingesta, almacenamiento, procesamiento, explotación, gobierno y seguridad. quizás en otro momento de mañana AM podamos avanzar con otra capa de la arquitectura.
6. todo esta PoC es de caracter academico, recordar conceptos, complejidades, metodologias, actualizar algunas herramientas que realizar cierta tarea.
7. recuerdo haber realizado una PoC donde tenia la ayuda de "oracle golden gate" para tomar datos desde un motor y luego replicarlo en un cluster.

se puede armar una PoC con estos datos?

ChatGPT said:

Sí, **se puede construir una PoC coherente con esos requisitos**, pero para obtener un resultado visible en aproximadamente una hora debemos reducir el alcance de la primera iteración.

La meta razonable para esta sesión sería:

Levantar un clúster HDFS académico de tres nodos, crear dos o tres bases relacionales con un modelo común, extraer sus datos mediante JDBC y almacenarlos en HDFS, verificando integridad y distribución.

Ajuste conceptual importante

Aunque coloquialmente se habla de “un maestro y dos esclavos”, la terminología actual es:

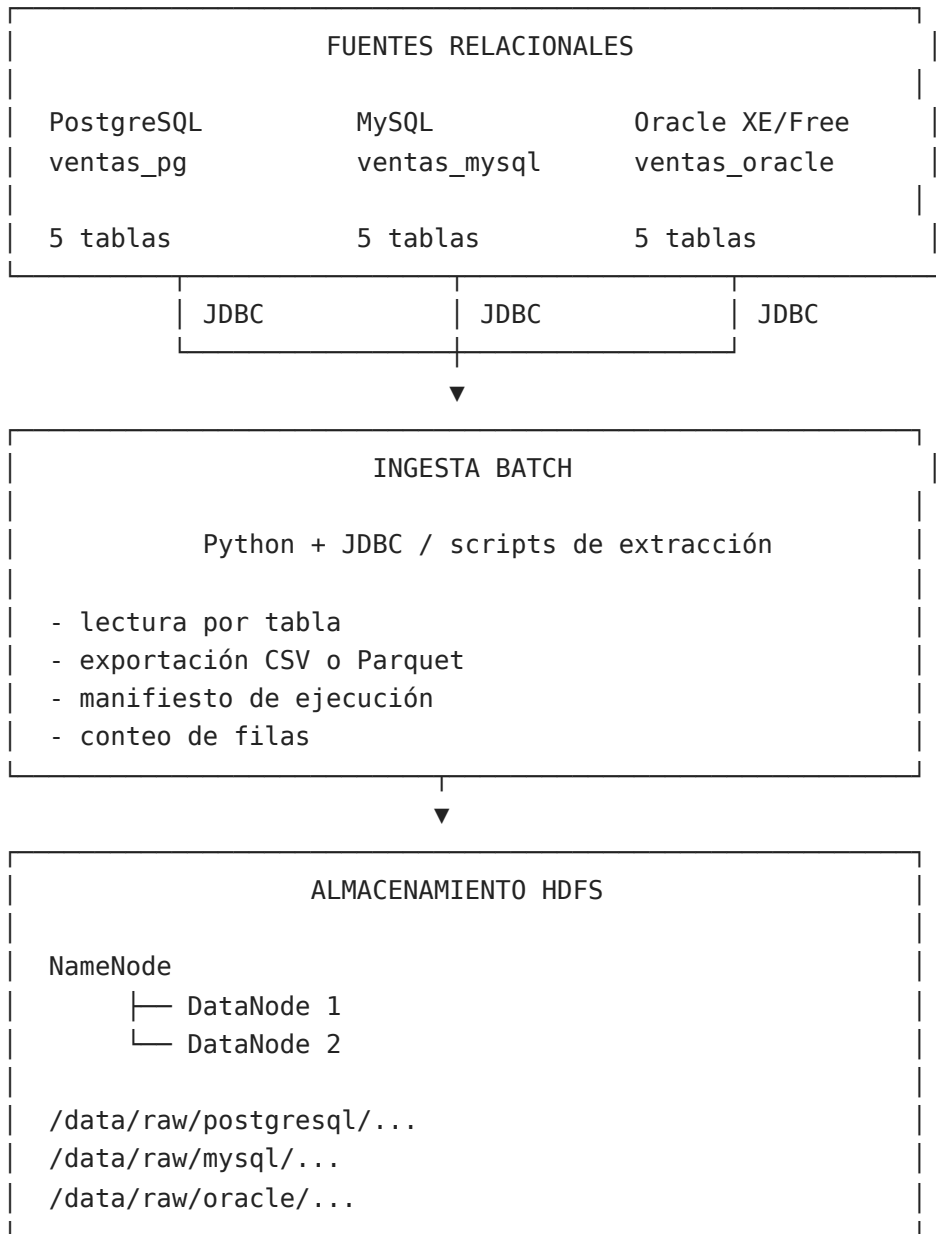
- **NameNode**: administra el espacio de nombres y los metadatos de HDFS.
- **DataNode 1 y DataNode 2**: almacenan físicamente los bloques.
- Opcionalmente, **ResourceManager** y **NodeManager** si incorporamos YARN y procesamiento.

En esta primera etapa no necesitamos un clúster Hadoop completo con todas sus capacidades. Basta con:

- 1 NameNode
- 2 DataNodes

Hadoop dispone actualmente de documentación oficial para levantar entornos mediante Compose y escalar DataNodes. 🍁 hadoop.apache.org

Arquitectura propuesta para la PoC v0.1



Esta versión cubre formalmente dos capas:

1. **Ingesta.**
2. **Almacenamiento distribuido.**

Deja preparadas las rutas para mañana incorporar:

3. Procesamiento.
4. Explotación.
5. Gobierno y seguridad.

Qué haría durante la primera hora

Resultado mínimo viable

Al finalizar deberíamos poder ejecutar algo similar a:

```
podman compose up -d
```

Y posteriormente comprobar:

```
hdfs dfs -ls -R /data/raw
```

Con una salida conceptual como:

```
/data/raw/mysql/clientes/  
/data/raw/mysql/productos/  
/data/raw/mysql/ventas/  
/data/raw/postgresql/clientes/  
/data/raw/postgresql/productos/  
/data/raw/postgresql/ventas/  
/data/raw/oracle/clientes/  
/data/raw/oracle/productos/  
/data/raw/oracle/ventas/
```

Además, deberíamos verificar:

```
hdfs dfs -count /data/raw/mysql/ventas  
hdfs fsck /data/raw -files -blocks -locations
```

Esto demostraría que:

- las bases están operativas;
- los datos fueron generados;
- la ingesta se ejecutó;
- los archivos llegaron a HDFS;
- los bloques están distribuidos entre dos DataNodes.

Modelo de datos recomendado

En vez de utilizar esquemas de ejemplo diferentes en cada motor, recomiendo crear **el mismo modelo lógico en los tres motores**.

Por ejemplo:

```
clientes  
productos  
sucursales
```

ventas
detalle_ventas

Relaciones:

clientes 1 — N ventas
sucursales 1 — N ventas
ventas 1 — N detalle_ventas
productos 1 — N detalle_ventas

Esto permite comparar:

- tipos de datos;
- dialectos SQL;
- conectividad JDBC;
- rendimiento de extracción;
- conteos entre origen y destino;
- normalización de esquemas;
- tratamiento de fechas y decimales.

Podemos poblar cada motor con datos sintéticos equivalentes:

Tabla	Filas iniciales sugeridas
clientes	500
productos	100
sucursales	10
ventas	5.000
detalle_ventas	10.000–20.000

Es suficientemente pequeño para desplegar rápido, pero bastante grande para observar particiones, archivos y validaciones.

Herramienta de ingesta

Para esta primera versión recomiendo **Python**, no GoldenGate ni Debezium.

Motivo

GoldenGate corresponde a una solución de captura y replicación transaccional más avanzada. Oracle mantiene integraciones para enviar cambios hacia HDFS y Kafka, pero requiere componentes específicos y consideraciones de licenciamiento. La licencia de destino de GoldenGate para Big Data no incluye automáticamente funcionalidad de origen.  docs.oracle.com

Debezium es una alternativa moderna para CDC y dispone de conectores para MySQL, PostgreSQL y Oracle.  debezium.io Sin embargo, incorporarlo ahora implicaría:

- Kafka o Redpanda;
- Kafka Connect;
- configuración de logs transaccionales;
- permisos especiales en cada base;
- conectores JDBC y CDC;
- formato de eventos;
- sinks hacia HDFS.

Eso excede el objetivo de una hora.

Ingesta v0.1

Python

- └─ psycopg para PostgreSQL
- └─ mysql-connector-python para MySQL
- └─ python-oracledb para Oracle
- └─ pandas o pyarrow
- └─ cliente WebHDFS o comando `hdfs dfs -put`

Flujo:

`SELECT * FROM tabla`

↓

DataFrame

↓

Parquet o CSV

↓

HDFS `/data/raw/<motor>/<tabla>/fecha_carga=YYYY-MM-DD/`

Por qué no recomiendo Apache Sqoop

Sqoop era precisamente la herramienta clásica para importar datos desde bases relacionales hacia HDFS mediante MapReduce. Sin embargo, el proyecto fue retirado y trasladado al

Apache Attic; su última versión estable fue 1.4.7. ➤ sqoop.apache.org

Podría instalarse con fines históricos o académicos, pero no sería una buena base para una arquitectura nueva en 2026.

Podemos incluirlo posteriormente como ejercicio comparativo:

Enfoque clásico:

RDBMS → Sqoop → HDFS

Enfoque actual:

RDBMS → Airbyte/NiFi/Python → HDFS u Object Storage

Enfoque CDC:

RDBMS → Debezium/GoldenGate → Kafka → almacenamiento

Consideración sobre Oracle

Oracle será el componente más pesado y probablemente el que más tiempo consuma.

Para una ejecución en tu laptop corporativa con Podman hay dos alternativas:

Alternativa A — primera hora

PostgreSQL + MySQL + HDFS

Ventajas:

- imágenes más ligeras;
- inicialización rápida;
- menor consumo de memoria;
- menor riesgo de problemas con el registro de Oracle;
- permite validar la arquitectura completa.

Oracle se agrega en la versión v0.2.

Alternativa B — incluir Oracle desde el inicio

PostgreSQL + MySQL + Oracle Free + HDFS

Riesgos:

- imagen más grande;
- mayor memoria;
- arranque más lento;

- configuración de health check más compleja;
- posibilidad de consumir la hora solamente descargando e inicializando Oracle.

Mi recomendación es la alternativa A para hoy.

Distribución estimada de recursos

Para tu equipo con 32 GB de RAM:

Contenedor	Memoria estimada para laboratorio
NameNode	1 GB
DataNode 1	1–2 GB
DataNode 2	1–2 GB
PostgreSQL	512 MB–1 GB
MySQL	512 MB–1 GB
Ingesta Python	512 MB
Oracle posterior	2–4 GB

Total inicial:

5–8 GB aproximadamente

Debemos comprobar primero cuánta memoria tiene asignada `podman machine`, porque los 32 GB físicos del equipo no necesariamente están disponibles dentro de la máquina virtual de Podman.

Estructura sugerida del proyecto

En tu ruta corporativa:

```
U:\Proyectos\podman\bigdata-poc
|
├── compose.yml
├── .env
├── .env.example
├── README.md
|
└── config
```



```
"fecha_inicio": "2026-07-24T23:15:00-04:00",  
"fecha_fin": "2026-07-24T23:15:04-04:00",  
"filas_origen": 5000,  
"filas_extraidas": 5000,  
"archivo": "/data/raw/postgresql/ventas/fecha_carga=2026-07-24/ventas.parquet",  
"estado": "OK"  
}
```

Esto introduce desde la primera versión algunos principios de gobierno:

- procedencia;
- fecha de carga;
- cantidad de registros;
- estado;
- ubicación;
- identificador de ejecución.

Evolución posterior

La arquitectura puede crecer de forma controlada:

v0.1 — hoy

MySQL + PostgreSQL

↓

Python batch

↓

HDFS: 1 NameNode + 2 DataNodes

v0.2 — próxima sesión

+ Oracle Free

+ validación cruzada

+ cargas incrementales por timestamp

v0.3

HDFS

↓

Spark

↓

zona curated en Parquet

v0.4

Hive Metastore o Trino

↓

consultas SQL

↓

Superset

v0.5

MySQL/PostgreSQL/Oracle

↓

Debezium

↓

Kafka

↓

HDFS

Debezium permitiría capturar cambios desde los tres motores con una estructura homogénea de eventos.  debezium.io

v0.6

gobierno + seguridad

Incluyendo:

- catálogo;
- linaje;
- calidad;
- RBAC;
- secretos;
- cifrado;
- auditoría;
- políticas de retención.

Recomendación ejecutiva

Sí, la PoC es viable. Para obtener algo funcional rápidamente, fijaría este alcance:

PoC Big Data v0.1: ingesta batch desde PostgreSQL y MySQL hacia un clúster HDFS de un NameNode y dos DataNodes, utilizando Podman Compose, datos sintéticos comunes, almacenamiento particionado y validación mediante conteos y HDFS FSCK.

Oracle y CDC quedarían explícitamente planificados para la segunda iteración. No usaría Sqoop como componente principal por encontrarse retirado, ni GoldenGate en esta etapa debido al peso operativo y de licenciamiento. Hadoop recomienda Kerberos para clústeres productivos; el laboratorio inicial quedaría sin Kerberos y debe considerarse estrictamente académico y aislado. 🌸 hadoop.apache.org

El siguiente paso concreto es generar el proyecto `bigdata-poc-v0.1` con su `compose.yml`, configuración HDFS, bases de ejemplo, scripts de ingesta y prueba de humo.

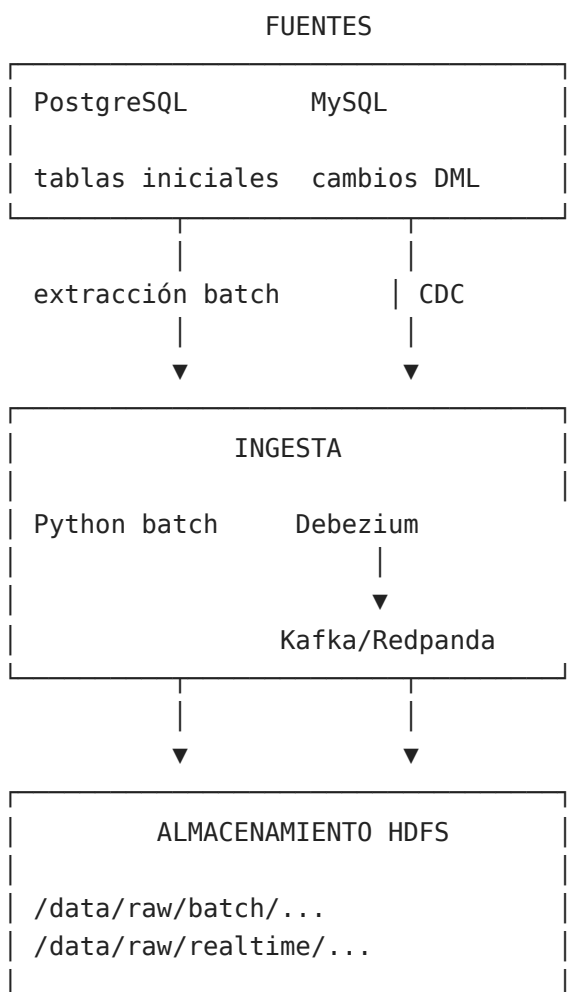
You said:

Podemos considerar en esta PoC v0.1 una ingesta batch y otra ingesta real time?

ChatGPT said:

Sí, podemos incluir ambas modalidades en la **PoC v0.1**, pero conviene tratarlas como dos pipelines independientes que convergen en el mismo almacenamiento HDFS.

La arquitectura quedaría así:



| NameNode + 2 DataNodes |

Kafka Connect está precisamente orientado a integrar Kafka con sistemas externos mediante conectores source y sink. Un sink hacia HDFS puede exportar registros desde tópicos Kafka hacia archivos en HDFS. 🌸 kafka.apache.org

Alcance recomendable para la v0.1

Para no sobredimensionar la primera iteración, propondría:

Flujo	Fuente	Mecanismo	Destino
Batch	PostgreSQL y MySQL	Python/JDBC	HDFS
Tiempo real	PostgreSQL o MySQL	Debezium → Kafka	HDFS
Almacenamiento	Ambos flujos	archivos CSV, JSON o Parquet	1 NameNode + 2 DataNodes

No intentaría implementar CDC simultáneamente para PostgreSQL, MySQL y Oracle en esta primera versión. Cada motor requiere configuraciones transaccionales distintas:

- PostgreSQL: replicación lógica, usuario con privilegios y publication/slot.
- MySQL: binary log, formato ROW e identificadores de servidor.
- Oracle: redo/archive logs, privilegios y configuración adicional.

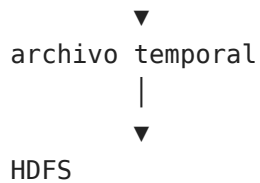
La v0.1 debería validar el patrón completo con **un solo motor en tiempo real**, idealmente PostgreSQL.

Diseño funcional de ambos pipelines

1. Ingesta batch

El proceso batch realizará una extracción completa:

```
PostgreSQL / MySQL
|
▼
SELECT * FROM tabla
|
```



Ejemplo de rutas:

/data/raw/batch/postgresql/clientes/fecha_carga=2026-07-25/
/data/raw/batch/postgresql/ventas/fecha_carga=2026-07-25/

/data/raw/batch/mysql/productos/fecha_carga=2026-07-25/
/data/raw/batch/mysql/sucursales/fecha_carga=2026-07-25/

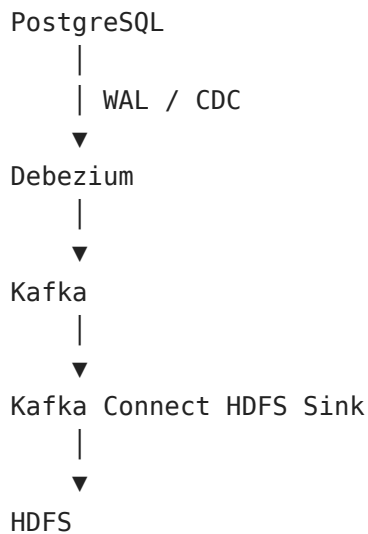
El batch representa el estado completo de una tabla en un momento determinado.

2. Ingesta en tiempo real

El pipeline en tiempo real capturará eventos generados por operaciones:

INSERT
UPDATE
DELETE

Flujo:



Ejemplo de rutas:

/data/raw/realtime/postgresql/clientes/
/data/raw/realtime/postgresql/ventas/

Cada evento debería conservar como mínimo:

```
{
  "operacion": "u",
  "timestamp_evento": "2026-07-25T23:40:12-04:00",
  "tabla": "clientes",
  "clave": 1001,
  "antes": {
    "estado": "ACTIVO"
  },
  "despues": {
    "estado": "SUSPENDIDO"
  }
}
```

Diferencia conceptual relevante

La ingesta real time no debería escribirse directamente como “una fila actualizada” sobre un archivo HDFS existente.

HDFS está orientado principalmente a:

- archivos grandes;
- escrituras secuenciales;
- lectura distribuida;
- almacenamiento de bloques;
- operaciones append controladas.

No es una base de datos transaccional para actualizar filas individuales.

Por eso, los cambios CDC se almacenan como un **log de eventos inmutable**:

INSERT → nuevo evento

UPDATE → nuevo evento

DELETE → nuevo evento

Posteriormente, en la capa de procesamiento, Spark podría reconstruir el estado vigente de cada entidad.

El clúster HDFS mantendría un NameNode para metadatos y DataNodes para almacenar los bloques físicos. 🌸 hadoop.apache.org

Tecnologías propuestas

Componentes mínimos

Fuentes

- └─ PostgreSQL
- └─ MySQL

Batch

- └─ Python

Tiempo real

- └─ Debezium PostgreSQL Connector
- └─ Kafka
- └─ Kafka Connect

Almacenamiento

- └─ HDFS NameNode
- └─ HDFS DataNode 1
- └─ HDFS DataNode 2

Esto implica aproximadamente nueve contenedores:

- 1 PostgreSQL
- 1 MySQL
- 1 NameNode
- 2 DataNodes
- 1 Kafka
- 1 Kafka Connect
- 1 Debezium Connect o plugin Debezium
- 1 contenedor de ingesta batch

Dependiendo de la distribución de Kafka, Debezium puede ejecutarse dentro de Kafka Connect, evitando un contenedor adicional.

Kafka o Redpanda

Para una PoC académica enfocada en conceptos Big Data, preferiría **Apache Kafka** porque permite estudiar explícitamente:

- brokers;
- tópicos;
- particiones;
- offsets;
- consumer groups;

- Kafka Connect;
- entrega de eventos;
- recuperación.

Redpanda consume menos recursos y simplifica el despliegue, pero ocultaría parcialmente algunos elementos tradicionales del ecosistema Kafka.

Kafka Connect estandariza la integración entre Kafka y sistemas externos, y los sink connectors consumen registros desde tópicos para escribirlos en el almacenamiento de destino. 🌸 kafka.apache.org

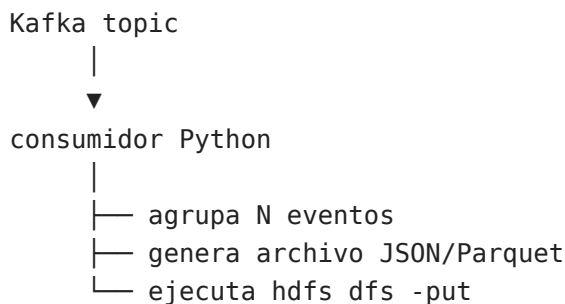
Restricción del sink hacia HDFS

Existe un conector HDFS 3 para Kafka Connect que escribe datos desde tópicos Kafka hacia HDFS y soporta diversos formatos. 🌐 docs.confluent.io

No obstante, existen dos consideraciones:

1. El conector HDFS 3 comúnmente utilizado pertenece al ecosistema Confluent.
2. Debemos revisar sus condiciones de licencia y compatibilidad antes de incorporarlo como dependencia permanente.

Una alternativa completamente controlable para la PoC es desarrollar un consumidor Python:



Esto sería técnicamente una ingesta **near real time o microbatch**, por ejemplo, escribiendo cada 10 segundos o cada 1.000 eventos.

Alternativas para la ruta real time

Opción	Flujo	Complejidad	Clasificación
A	Debezium → Kafka → HDFS Sink	Alta	Real time

Opción	Flujo	Complejidad	Clasificación
B	Debezium → Kafka → consumidor Python → HDFS	Media	Near real time
C	Script consulta cambios cada 10 segundos → HDFS	Baja	Microbatch
D	Aplicación genera eventos directamente en Kafka	Media	Real time simulado

Recomendación para v0.1

Usaría la opción **B**:

```

PostgreSQL
  ↓ CDC
Debezium
  ↓
Kafka
  ↓
Consumidor Python
  ↓ cada 10–30 segundos
HDFS

```

Ventajas:

- demuestra CDC real desde la fuente;
- evita depender inicialmente del sink HDFS de terceros;
- permite ver y controlar los eventos;
- facilita logs, métricas y validaciones;
- reduce problemas de compatibilidad entre versiones;
- permite evolucionar posteriormente a un conector especializado.

La denominación metodológicamente correcta sería:

Captura de cambios en tiempo real y persistencia en HDFS mediante microbatch de baja latencia.

No deberíamos afirmar que la escritura final en HDFS es estrictamente registro a registro en tiempo real.

Caso demostrativo

El modelo común puede contener:

clientes
productos
sucursales
ventas
detalle_ventas

Prueba batch

1. Poblar PostgreSQL y MySQL.
2. Extraer cinco tablas.
3. Escribir archivos en HDFS.
4. Comparar conteos origen/destino.

Prueba real time

1. Insertar un cliente.
2. Modificar un producto.
3. Crear una venta.
4. Eliminar o anular un registro.
5. Ver los eventos en Kafka.
6. Comprobar su llegada a HDFS.

Ejemplo:

```
INSERT INTO clientes (  
    cliente_id,  
    nombre,  
    estado  
)  
VALUES (  
    1001,  
    'Cliente CDC',  
    'ACTIVO'  
);
```

```
UPDATE clientes  
SET estado = 'SUSPENDIDO'  
WHERE cliente_id = 1001;
```

Validación conceptual:


```
kafka-console-consumer \  
  --bootstrap-server kafka:9092 \  
  --topic postgres.public.clientes \  
  --from-beginning
```

Y luego:

```
hdfs dfs -ls -R /data/raw/realtime/postgresql/clientes  
hdfs dfs -cat /data/raw/realtime/postgresql/clientes/*.json
```

Alcance formal de la PoC v0.1

La definiría de esta forma:

Implementar una plataforma contenerizada de ingesta batch y captura de cambios en tiempo real desde motores relacionales hacia almacenamiento distribuido HDFS. La ruta batch extraerá datos completos desde PostgreSQL y MySQL. La ruta de tiempo real capturará cambios desde PostgreSQL mediante Debezium y Kafka, consolidándolos periódicamente en HDFS.

Qué dejaría fuera de la v0.1

Para mantenerla abordable:

- Oracle como fuente CDC.
- CDC simultáneo de los tres motores.
- Spark.
- YARN.
- Hive.
- Trino.
- Superset.
- catálogo de datos.
- Kerberos.
- alta disponibilidad del NameNode.
- Schema Registry.
- semántica exactly-once integral.
- resolución automática de duplicados.

Evolución sugerida

v0.1

└─ batch PostgreSQL y MySQL
└─ CDC PostgreSQL

- └─ Kafka
- └─ consumidor HDFS
- └─ HDFS 1+2

v0.2

- └─ CDC MySQL
- └─ formatos Parquet/Avro
- └─ control de offsets
- └─ idempotencia

v0.3

- └─ Oracle
- └─ comparación Debezium/GoldenGate
- └─ validación multifuente

v0.4

- └─ Spark
- └─ reconstrucción de estados
- └─ capa curated

v0.5

- └─ Trino/Hive
- └─ visualización
- └─ explotación

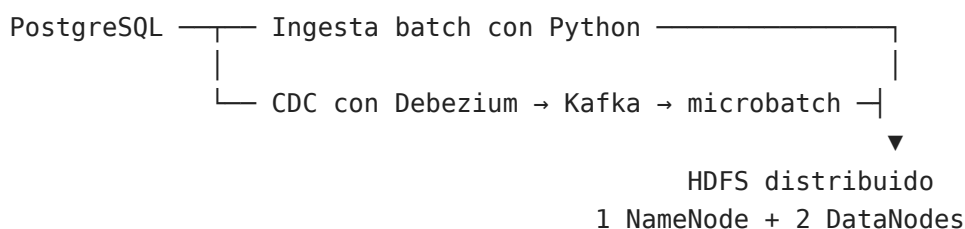
Por tanto, **sí**, ambas ingestas pueden incorporarse desde la v0.1. La condición para que sea realizable es limitar el tiempo real a PostgreSQL y considerar la persistencia en HDFS como microbatch, manteniendo Kafka como transporte de eventos en tiempo real.

You said:

Ok, estoy de acuerdo con el plan. tengo windows 11 pro, tengo podman instalado ya, vscode y python. adelante, te sigo

ChatGPT said:

Perfecto. Avanzaremos de forma incremental, validando cada etapa antes de agregar componentes. La **PoC v0.1** tendrá inicialmente:



MySQL se incorporará una vez validado el primer flujo completo. Esto reduce variables durante el diagnóstico.

En Windows, Podman ejecuta los contenedores Linux dentro de una máquina virtual denominada **Podman Machine**. Además, `podman compose` utiliza un proveedor Compose externo, por lo que primero verificaremos ambos componentes.  podman-desktop.io  podman.io

Etapa 0: validar el entorno

Abre **PowerShell** desde VS Code o Windows Terminal. No necesitas permisos de administrador.

1. Verificar Podman

Ejecuta:

```
podman version
```

Luego:

```
podman machine list
```

El resultado esperado debería mostrar una máquina con estado `Currently running` o equivalente.

Si aparece detenida:

```
podman machine start
```

Después valida la comunicación:

```
podman info
```

No continúes si `podman info` devuelve un error de conexión.

2. Verificar Compose

Ejecuta:

```
podman compose version
```

Resultado esperado:

```
podman-compose version ...
```

O:

Docker Compose version ...

Podman puede utilizar distintos proveedores compatibles con Compose; lo importante es que el comando termine correctamente. Podman Desktop también permite instalar o configurar el motor Compose desde **Settings** → **Resources** → **Compose**. podman-desktop.io

Si el comando falla, ejecuta adicionalmente:

```
docker-compose version
```

Por ahora no instales nada manualmente. Primero necesito ver el mensaje exacto.

3. Revisar recursos de Podman Machine

Ejecuta:

```
podman machine inspect
```

Busca en la salida valores relacionados con:

CPU
Memory
DiskSize
Rootful

Para la PoC completa recomiendo como mínimo:

Recurso de Podman Machine	Mínimo	Recomendado
CPU	4 vCPU	6–8 vCPU
Memoria	8 GB	12–16 GB
Disco	30 GB	50–80 GB

La versión inicial utilizará aproximadamente:

- PostgreSQL: 0,5–1 GB.
- Kafka: 1–2 GB.
- Kafka Connect/Debezium: 1–2 GB.
- NameNode: 1 GB.
- Dos DataNodes: 1–2 GB cada uno.
- Contenedor Python: menos de 1 GB.

No significa que todos consumirán simultáneamente el límite máximo, pero una máquina de solo 4 GB será insuficiente.

4. Validar ejecución de contenedores

Ejecuta una prueba no destructiva:

```
podman run --rm docker.io/library/alpine:3.22 `
  sh -c "echo 'Podman operativo'; uname -a; cat /etc/os-release"
```

Resultado esperado:

```
Podman operativo
Linux ...
NAME="Alpine Linux"
...
```

Este comando descargará una imagen pequeña y la eliminará después de ejecutar la prueba.

Etapas 1: crear el directorio del proyecto

Utilizaremos tu ruta:

```
U:\Proyectos\podman\bigdata-poc
```

Ejecuta:

```
$RaizProyecto = "U:\Proyectos\podman\bigdata-poc"
```

```
New-Item -ItemType Directory -Path $RaizProyecto -Force |
  Out-Null
```

```
Set-Location $RaizProyecto
```

```
Get-Location
```

Después crea la estructura inicial:

```
$Directorios = @(
  "config\hadoop",
  "database\postgresql",
  "database\mysql",
  "database\oracle",
  "ingestion\batch\src",
  "ingestion\realtime\src",
```

```

        "connectors\debezium",
        "scripts",
        "data\staging",
        "logs",
        "manifests"
    )

    foreach ($Directorio in $Directorios) {
        New-Item `
            -ItemType Directory `
            -Path (Join-Path $RaizProyecto $Directorio) `
            -Force |
            Out-Null
    }

    Get-ChildItem -Path $RaizProyecto -Recurse -Directory |
        Select-Object FullName

```

La estructura resultante será aproximadamente:

```

bigdata-poc
├── config
│   └── hadoop
├── connectors
│   └── debezium
├── database
│   ├── mysql
│   ├── oracle
│   └── postgresql
├── ingestion
│   ├── batch
│   │   └── src
│   └── realtime
│       └── src
├── scripts
├── data
│   └── staging
├── logs
└── manifests

```

Etapla 2: validar la ruta U:

Esta comprobación es importante porque U: podría ser:

- una unidad local;
- una unidad corporativa mapeada por red;

- una unidad sincronizada;
- una ruta con restricciones de montaje dentro de Podman Machine.

Crea un archivo de prueba:

```
"prueba-montaje-$(Get-Date -Format 'yyyyMMdd-HH:mm:ss') " |
    Set-Content -Path ".\data\staging\prueba.txt" `
        -Encoding UTF8
```

Luego intenta montarlo dentro de un contenedor:

```
$RutaStaging = (Resolve-Path ".\data\staging").Path
```

```
podman run --rm `
    --volume "${RutaStaging}:/datos:ro" `
    docker.io/library/alpine:3.22 `
    sh -c "echo 'Contenido montado:'; ls -la /datos; cat /datos/prueba.txt"
```

Debe aparecer el contenido de prueba.txt.

Interpretación

Caso A: funciona

Podemos mantener:

```
U:\Proyectos\podman\bigdata-poc
```

Caso B: aparece **permission denied**, ruta inexistente o montaje vacío

No construiremos todavía la solución sobre U:. Probaremos temporalmente una ruta local:

```
$RutaAlternativa = "C:\podman-data\bigdata-poc-test"
```

```
New-Item -ItemType Directory `
    -Path $RutaAlternativa `
    -Force |
    Out-Null
```

Esto es relevante porque HDFS, PostgreSQL y Kafka necesitan persistencia fiable. Una unidad corporativa de red puede presentar:

- latencia elevada;
- problemas de permisos;
- incompatibilidad con enlaces o atributos Linux;
- desconexiones;

- menor rendimiento de escritura;
- fallos al reiniciar Podman Machine.

Podemos mantener el código fuente en U: y almacenar los volúmenes internos como **volúmenes administrados por Podman**, evitando escribir directamente los datos de HDFS sobre la unidad Windows.

Resultado que necesito

Ejecuta los siguientes comandos y comparte su salida:

```
podman version
podman machine list
podman compose version
podman machine inspect
```

También indícame si terminaron correctamente estas dos pruebas:

```
podman run --rm docker.io/library/alpine:3.22 `
  sh -c "echo 'Podman operativo'; uname -a"

$RutaStaging = (Resolve-Path `
  "U:\Proyectos\podman\bigdata-poc\data\staging").Path

podman run --rm `
  --volume "${RutaStaging}:/datos:ro" `
  docker.io/library/alpine:3.22 `
  sh -c "ls -la /datos; cat /datos/prueba.txt"
```

Con esa validación, el siguiente bloque será construir la **Etapla 1 funcional**: PostgreSQL con cinco tablas, datos sintéticos reproducibles y health check; después levantaremos HDFS.

ChatGPT said:

Bien. Las validaciones principales resultaron correctas:

- Podman 5.8.5 está operativo.
- La máquina WSL está en ejecución.
- Tienes 8 CPU asignadas.
- Docker Compose v5.3.1 está siendo utilizado como proveedor externo.
- El montaje desde Windows hacia el contenedor funciona correctamente.
- La ruta actual del proyecto es:

C:\rutinas-local\gen-ai-app\bigdata-poc

El único punto que debemos corregir antes de levantar Hadoop, Kafka y Debezium es la memoria: actualmente WSL dispone de **4 GiB**, insuficiente para ejecutar la PoC completa de forma estable.

Paso 1: ampliar memoria de WSL

En PowerShell ejecuta:

```
notepad "$env:USERPROFILE\.wslconfig"
```

Agrega este contenido:

```
[wsl2]
memory=12GB
processors=8
swap=4GB
```

Guarda el archivo.

Esta configuración se aplica globalmente a las distribuciones WSL 2, incluida la máquina utilizada por Podman. Microsoft documenta %UserProfile%.wslconfig como el archivo destinado a controlar memoria, CPU y otros recursos de la máquina virtual WSL 2. 

learn.microsoft.com

Impacto

El comando siguiente detendrá temporalmente:

- Podman Machine;
- otras distribuciones WSL abiertas;
- procesos que estén ejecutándose dentro de WSL.

No elimina imágenes, volúmenes ni contenedores.

Ejecuta:

```
podman machine stop
wsl --shutdown
```

Espera aproximadamente 10 segundos y luego inicia nuevamente:

```
podman machine start
```

Valida los recursos reales disponibles dentro de la máquina:

```
podman machine ssh free -h
podman machine ssh nproc
podman machine ssh "df -h /"
```

El resultado esperado de `free -h` debería mostrar aproximadamente:

```
Mem: 11Gi - 12Gi
Swap: 4Gi
```

La documentación oficial de Podman indica que en Windows los contenedores Linux se ejecutan mediante Podman Machine; en el backend WSL la configuración global de recursos se administra en WSL. docs.podman.io learn.microsoft.com

Es posible que `podman machine list` continúe mostrando el valor registrado originalmente. Para esta validación, la referencia efectiva será `free -h` ejecutado dentro de la máquina.

Paso 2: preparar la primera fuente PostgreSQL

Mientras validamos los recursos, comenzaremos con un componente aislado:

```
PostgreSQL
├─ esquema comercial
│   ├── clientes
│   ├── productos
│   ├── sucursales
│   ├── ventas
│   └─ detalle_ventas
```

Sitúate en el proyecto:

```
Set-Location "C:\rutinas-local\gen-ai-app\bigdata-poc"
```

Crea los directorios necesarios:

```
$Directorios = @(
    "database\postgresql\init",
    "config\hadoop",
    "ingestion\batch\src",
    "ingestion\realtime\src",
    "scripts",
    "logs",
    "manifests"
)

foreach ($Directorio in $Directorios) {
    New-Item -ItemType Directory -Path $Directorio -Force |
```

```
        Out-Null
    }
}
```

Archivo .env

Crea el archivo:

```
notepad .env
```

Contenido:

```
POSTGRES_DB=ventas
POSTGRES_USER=bigdata
POSTGRES_PASSWORD=BigData_PoC_2026
POSTGRES_PORT=5432
```

Esta contraseña es aceptable únicamente para el laboratorio local. El archivo .env no debe publicarse en Git.

Crea también:

```
notepad .gitignore
```

Contenido:

```
.env
logs/*
manifests/*
data/*
!.gitkeep
__pycache__/*
*.pyc
```

Paso 3: crear el modelo relacional

Crea:

```
notepad .\database\postgresql\init\01_esquema.sql
```

Contenido:

```
CREATE TABLE sucursales (
    sucursal_id INTEGER PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    ciudad VARCHAR(100) NOT NULL,
    activa BOOLEAN NOT NULL DEFAULT TRUE
);
```

```

CREATE TABLE clientes (
    cliente_id INTEGER PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL,
    correo VARCHAR(150) NOT NULL UNIQUE,
    fecha_registro DATE NOT NULL,
    estado VARCHAR(20) NOT NULL
        CHECK (estado IN ('ACTIVO', 'INACTIVO', 'SUSPENDIDO'))
);

CREATE TABLE productos (
    producto_id INTEGER PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL,
    categoria VARCHAR(100) NOT NULL,
    precio NUMERIC(12,2) NOT NULL CHECK (precio >= 0),
    stock INTEGER NOT NULL CHECK (stock >= 0),
    activo BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE ventas (
    venta_id BIGINT PRIMARY KEY,
    cliente_id INTEGER NOT NULL,
    sucursal_id INTEGER NOT NULL,
    fecha_venta TIMESTAMP NOT NULL,
    estado VARCHAR(20) NOT NULL
        CHECK (estado IN ('CREADA', 'PAGADA', 'ANULADA')),
    total NUMERIC(14,2) NOT NULL CHECK (total >= 0),

    CONSTRAINT fk_ventas_cliente
        FOREIGN KEY (cliente_id)
        REFERENCES clientes(cliente_id),

    CONSTRAINT fk_ventas_sucursal
        FOREIGN KEY (sucursal_id)
        REFERENCES sucursales(sucursal_id)
);

CREATE TABLE detalle_ventas (
    detalle_id BIGINT PRIMARY KEY,
    venta_id BIGINT NOT NULL,
    producto_id INTEGER NOT NULL,
    cantidad INTEGER NOT NULL CHECK (cantidad > 0),
    precio_unitario NUMERIC(12,2) NOT NULL CHECK (precio_unitario >= 0),
    subtotal NUMERIC(14,2) NOT NULL CHECK (subtotal >= 0),

    CONSTRAINT fk_detalle_venta
        FOREIGN KEY (venta_id)
        REFERENCES ventas(venta_id),

    CONSTRAINT fk_detalle_producto

```

```

        FOREIGN KEY (producto_id)
        REFERENCES productos(producto_id)
    );

CREATE INDEX idx_ventas_cliente
    ON ventas(cliente_id);

CREATE INDEX idx_ventas_fecha
    ON ventas(fecha_venta);

CREATE INDEX idx_detalle_venta
    ON detalle_ventas(venta_id);

```

Paso 4: poblar datos reproducibles

Crea:

notepad .\database\postgresql\init\02_datos.sql

Contenido:

```

INSERT INTO sucursales (
    sucursal_id,
    nombre,
    ciudad,
    activa
)
SELECT
    numero,
    'Sucursal ' || numero,
    CASE
        WHEN numero <= 3 THEN 'Santiago'
        WHEN numero <= 5 THEN 'Valparaíso'
        WHEN numero <= 7 THEN 'Concepción'
        ELSE 'Rancagua'
    END,
    TRUE
FROM generate_series(1, 10) AS numero;

INSERT INTO clientes (
    cliente_id,
    nombre,
    correo,
    fecha_registro,
    estado
)
SELECT

```

```

    numero,
    'Cliente ' || numero,
    'cliente' || numero || '@poc.local',
    DATE '2024-01-01' + ((numero - 1) % 730),
    CASE
        WHEN numero % 20 = 0 THEN 'SUSPENDIDO'
        WHEN numero % 10 = 0 THEN 'INACTIVO'
        ELSE 'ACTIVO'
    END
FROM generate_series(1, 500) AS numero;

```

```

INSERT INTO productos (
    producto_id,
    nombre,
    categoria,
    precio,
    stock,
    activo
)
SELECT
    numero,
    'Producto ' || numero,
    CASE
        WHEN numero % 4 = 0 THEN 'Computación'
        WHEN numero % 4 = 1 THEN 'Electrónica'
        WHEN numero % 4 = 2 THEN 'Accesorios'
        ELSE 'Oficina'
    END,
    ROUND((1000 + numero * 137.25)::NUMERIC, 2),
    10 + ((numero * 7) % 200),
    TRUE
FROM generate_series(1, 100) AS numero;

```

```

INSERT INTO ventas (
    venta_id,
    cliente_id,
    sucursal_id,
    fecha_venta,
    estado,
    total
)
SELECT
    numero,
    ((numero - 1) % 500) + 1,
    ((numero - 1) % 10) + 1,
    TIMESTAMP '2025-01-01 08:00:00'
        + ((numero - 1) * INTERVAL '10 minutes'),

```

```

CASE
    WHEN numero % 30 = 0 THEN 'ANULADA'
    WHEN numero % 5 = 0 THEN 'CREADA'
    ELSE 'PAGADA'
END,
0
FROM generate_series(1, 5000) AS numero;

INSERT INTO detalle_ventas (
    detalle_id,
    venta_id,
    producto_id,
    cantidad,
    precio_unitario,
    subtotal
)
SELECT
    numero,
    ((numero - 1) % 5000) + 1,
    ((numero - 1) % 100) + 1,
    ((numero - 1) % 4) + 1,
    p.precio,
    ROUND(
        (
            (((numero - 1) % 4) + 1)
            * p.precio
        )::NUMERIC,
        2
    )
FROM generate_series(1, 15000) AS numero
JOIN productos p
    ON p.producto_id = ((numero - 1) % 100) + 1;

```

```

UPDATE ventas v
SET total = totales.total
FROM (
    SELECT
        venta_id,
        SUM(subtotal) AS total
    FROM detalle_ventas
    GROUP BY venta_id
) AS totales
WHERE v.venta_id = totales.venta_id;

```

El uso de `generate_series` permite crear datos deterministas dentro del propio PostgreSQL, sin agregar todavía dependencias Python.

Paso 5: crear compose.yml

Crea:

notepad compose.yml

Contenido inicial:

name: bigdata-poc

services:

postgresql:

image: docker.io/library/postgres:17

container_name: bigdata-postgresql

restart: unless-stopped

environment:

POSTGRES_DB: \${POSTGRES_DB}

POSTGRES_USER: \${POSTGRES_USER}

POSTGRES_PASSWORD: \${POSTGRES_PASSWORD}

ports:

- "\${POSTGRES_PORT}:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

- ./database/postgresql/init:/docker-entrypoint-initdb.d:ro

healthcheck:

test:

[

"CMD-SHELL",

"pg_isready -U \${POSTGRES_USER} -d \${POSTGRES_DB}"

]

interval: 10s

timeout: 5s

retries: 10

start_period: 20s

networks:

- bigdata_net

volumes:

postgres_data:

name: bigdata_postgres_data

networks:

bigdata_net:

name: bigdata_net

Estamos utilizando un volumen administrado por Podman para los archivos internos de PostgreSQL. Esto evita problemas de permisos y rendimiento al almacenar directamente el directorio de datos sobre NTFS.

Paso 6: validar y levantar PostgreSQL

Primero valida la sintaxis:

```
podman compose config
```

Si no aparecen errores:

```
podman compose up -d postgresql
```

Comprueba:

```
podman ps --filter "name=bigdata-postgresql"
```

Revisa el inicio:

```
podman logs --tail 100 bigdata-postgresql
```

Espera hasta que el estado sea saludable:

```
podman inspect bigdata-postgresql `
  --format "{{.State.Health.Status}}"
```

Resultado esperado:

```
healthy
```

Paso 7: prueba de humo

Ejecuta:

```
podman exec bigdata-postgresql `
  psql `
  -U bigdata `
  -d ventas `
  -c "\dt"
```

Luego valida los conteos:

```
podman exec bigdata-postgresql `
  psql `
  -U bigdata `
  -d ventas `
  -c @"
```

```

SELECT 'sucursales' AS tabla, COUNT(*) AS filas FROM sucursales
UNION ALL
SELECT 'clientes', COUNT(*) FROM clientes
UNION ALL
SELECT 'productos', COUNT(*) FROM productos
UNION ALL
SELECT 'ventas', COUNT(*) FROM ventas
UNION ALL
SELECT 'detalle_ventas', COUNT(*) FROM detalle_ventas
ORDER BY tabla;
"@

```

Resultado esperado:

Tabla	Filas
clientes	500
detalle_ventas	15.000
productos	100
sucursales	10
ventas	5.000

Nota sobre reinicialización

Los scripts de `/docker-entrypoint-initdb.d` solo se ejecutan cuando el volumen de PostgreSQL está vacío.

Durante esta fase de laboratorio, para reconstruir completamente la base:

```

podman compose down
podman volume rm bigdata_postgres_data
podman compose up -d postgresql

```

Esto es destructivo para los datos del laboratorio PostgreSQL. No se debe utilizar posteriormente cuando queramos conservar datos o probar CDC.

Completa primero la ampliación de WSL y el levantamiento de PostgreSQL. El próximo bloque será el clúster HDFS con **un NameNode y dos DataNodes**, usando volúmenes persistentes y una prueba `fsck` para verificar la ubicación de los bloques.

You said:

Listo con esta fase, continuemos

ChatGPT said:

Perfecto. Continuamos con la **fase de almacenamiento distribuido**, incorporando HDFS con:

1 NameNode
2 DataNodes
Factor de replicación: 2

Usaremos la imagen `apache/hadoop:3.5.0`, publicada por Apache Software Foundation. La imagen oficial permite iniciar los servicios mediante los comandos `hdfs namenode` y `hdfs datanode`, y admite configuración mediante variables que se traducen a propiedades de Hadoop. github.com hub.docker.com

Fase 2: clúster HDFS

1. Posicionarse en el proyecto

```
Set-Location "C:\rutinas-local\gen-ai-app\bigdata-poc"
```

Confirma:

```
Get-Location
```

2. Crear la configuración de Hadoop

Crea el archivo:

```
notepad .\config\hadoop\hadoop.env
```

Contenido:

```
CORE-SITE.XML_fs.defaultFS=hdfs://namenode:8020  
  
HDFS-SITE.XML_dfs.namenode.rpc-address=namenode:8020  
HDFS-SITE.XML_dfs.replication=2  
HDFS-SITE.XML_dfs.namenode.name.dir=file:///tmp/hadoop-root/dfs/name  
HDFS-SITE.XML_dfs.datanode.data.dir=file:///tmp/hadoop-root/dfs/data
```

HDFS-SITE.XML_df.permissions.enabled=false
HDFS-SITE.XML_df.namenode.datanode.registration.ip-hostname-check=false

Interpretación

Propiedad	Función
fs.defaultFS	Define el NameNode predeterminado
dfs.replication=2	Mantiene dos copias de cada bloque
dfs.namenode.name.dir	Persistencia de metadatos
dfs.datanode.data.dir	Persistencia de bloques
permissions.enabled=false	Simplifica el laboratorio
registration.ip-hostname-check=false	Evita problemas de validación de hostname en contenedores

La configuración oficial de ejemplo utiliza precisamente `hdfs://namenode`, el puerto RPC 8020 y variables con el formato `CORE-SITE.XML_*` y `HDFS-SITE.XML_*`. github.com

La desactivación de permisos es exclusivamente para esta PoC local. No corresponde a una configuración productiva.

3. Actualizar `compose.yml`

Reemplaza el contenido de `compose.yml` por la siguiente versión completa, que conserva PostgreSQL y agrega HDFS:

```
name: bigdata-poc

services:

  # =====
  # FUENTE DE DATOS: POSTGRESQL
  # =====
  postgresql:
    image: docker.io/library/postgres:17
    container_name: bigdata-postgresql
    hostname: postgresql
    restart: unless-stopped
```

```

environment:
  POSTGRES_DB: ${POSTGRES_DB}
  POSTGRES_USER: ${POSTGRES_USER}
  POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}

ports:
  - "${POSTGRES_PORT}:5432"

volumes:
  - postgres_data:/var/lib/postgresql/data
  - ./database/postgresql/init:/docker-entrypoint-initdb.d:ro

healthcheck:
  test:
    [
      "CMD-SHELL",
      "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"
    ]
  interval: 10s
  timeout: 5s
  retries: 10
  start_period: 20s

networks:
  - bigdata_net

```

```

# =====
# HDFS: NAMENODE
# =====
namenode:
  image: docker.io/apache/hadoop:3.5.0
  container_name: bigdata-hdfs-namenode
  hostname: namenode
  restart: unless-stopped

command:
  - hdfs
  - namenode

env_file:
  - ./config/hadoop/hadoop.env

environment:
  ENSURE_NAMENODE_DIR: /tmp/hadoop-root/dfs/name

ports:
  - "9870:9870"

```

```

volumes:
  - hdfs_namenode_data:/tmp/hadoop-root/dfs/name

networks:
  - bigdata_net

# =====
# HDFS: DATANODE 1
# =====
datanode1:
  image: docker.io/apache/hadoop:3.5.0
  container_name: bigdata-hdfs-datanode1
  hostname: datanode1
  restart: unless-stopped

  command:
    - hdfs
    - datanode

  env_file:
    - ./config/hadoop/hadoop.env

  depends_on:
    - namenode

  ports:
    - "9864:9864"

  volumes:
    - hdfs_datanode1_data:/tmp/hadoop-root/dfs/data

  networks:
    - bigdata_net

# =====
# HDFS: DATANODE 2
# =====
datanode2:
  image: docker.io/apache/hadoop:3.5.0
  container_name: bigdata-hdfs-datanode2
  hostname: datanode2
  restart: unless-stopped

  command:
    - hdfs
    - datanode

```

```
env_file:
  - ./config/hadoop/hadoop.env

depends_on:
  - namenode

ports:
  - "9865:9864"

volumes:
  - hdfs_datanode2_data:/tmp/hadoop-root/dfs/data

networks:
  - bigdata_net
```

```
volumes:
  postgres_data:
    name: bigdata_postgres_data

  hdfs_namenode_data:
    name: bigdata_hdfs_namenode_data

  hdfs_datanode1_data:
    name: bigdata_hdfs_datanode1_data

  hdfs_datanode2_data:
    name: bigdata_hdfs_datanode2_data

networks:
  bigdata_net:
    name: bigdata_net
```

El patrón command: ["hdfs", "namenode"] y command: ["hdfs", "datanode"] corresponde al compose publicado por el proyecto Apache Hadoop. github.com

4. Validar la configuración Compose

Ejecuta:

```
podman compose config
```

Revisa especialmente que aparezcan:

```
bigdata-hdfs-namenode
bigdata-hdfs-datanode1
bigdata-hdfs-datanode2
```

También verifica que la variable de replicación se haya cargado:

```
podman compose config |
  Select-String "dfs.replication"
```

Resultado esperado:

```
HDFS-SITE.XML_dfs.replication: "2"
```

5. Descargar la imagen Hadoop

La imagen es relativamente grande, por lo que esta etapa podría tardar algunos minutos:

```
podman pull docker.io/apache/hadoop:3.5.0
```

Comprueba:

```
podman images |
  Where-Object Repository -Match "apache/hadoop"
```

La versión 3.5.0 es la versión publicada por Apache en marzo de 2026 y su imagen de conveniencia está disponible en Docker Hub. 🌸 hadoop.apache.org 🐳 hub.docker.com

6. Levantar HDFS

Ejecuta:

```
podman compose up -d namenode datanode1 datanode2
```

Comprueba los contenedores:

```
podman ps --format `
  "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Deberías observar:

```
bigdata-hdfs-namenode
bigdata-hdfs-datanode1
bigdata-hdfs-datanode2
bigdata-postgresql
```

7. Revisar los logs

NameNode

```
podman logs --tail 100 bigdata-hdfs-namenode
```

Busca mensajes similares a:

NameNode RPC up

Web-server up

DataNode 1

```
podman logs --tail 80 bigdata-hdfs-datanode1
```

DataNode 2

```
podman logs --tail 80 bigdata-hdfs-datanode2
```

Busca registros relacionados con:

Block pool

Registered

DataNode

namenode

Para observar el proceso en tiempo real:

```
podman logs -f bigdata-hdfs-namenode
```

Presiona **Ctrl+C** para salir del seguimiento; el contenedor seguirá ejecutándose.

8. Verificar el estado del clúster

Espera aproximadamente 20–40 segundos y ejecuta:

```
podman exec bigdata-hdfs-namenode `
    hdfs dfsadmin -report
```

El resultado esperado debe contener:

Live datanodes (2)

Y dos bloques de información:

Name: datanode1

...

Name: datanode2

También puedes consultar únicamente la cantidad:

```
podman exec bigdata-hdfs-namenode `
  hdfs dfsadmin -report |
  Select-String "Live datanodes"
```

Resultado esperado:

Live datanodes (2)

9. Revisar la interfaz web

Abre en el navegador:

`http://localhost:9870`

En la interfaz del NameNode revisa:

Datanodes → Live Nodes

Deben aparecer:

datanode1
datanode2

Las interfaces individuales de los DataNodes estarán expuestas en:

DataNode 1: `http://localhost:9864`
DataNode 2: `http://localhost:9865`

El NameNode administra el espacio de nombres y los metadatos, mientras que los DataNodes almacenan físicamente los bloques. HDFS replica esos bloques entre los DataNodes según el factor configurado. 🌸 hadoop.apache.org

10. Crear la estructura de almacenamiento

Ejecuta:

```
podman exec bigdata-hdfs-namenode `
  hdfs dfs -mkdir -p /data/raw/batch/postgresql
```

Prepara las rutas futuras:

```
podman exec bigdata-hdfs-namenode `
    hdfs dfs -mkdir -p /data/raw/batch/mysql

podman exec bigdata-hdfs-namenode `
    hdfs dfs -mkdir -p /data/raw/batch/oracle

podman exec bigdata-hdfs-namenode `
    hdfs dfs -mkdir -p /data/raw/realtime/postgresql

podman exec bigdata-hdfs-namenode `
    hdfs dfs -mkdir -p /data/manifests

podman exec bigdata-hdfs-namenode `
    hdfs dfs -mkdir -p /data/checkpoints
```

Comprueba:

```
podman exec bigdata-hdfs-namenode `
    hdfs dfs -ls -R /data
```

Resultado conceptual:

```
/data/checkpoints
/data/manifests
/data/raw
/data/raw/batch
/data/raw/batch/mysql
/data/raw/batch/oracle
/data/raw/batch/postgresql
/data/raw/realtime
/data/raw/realtime/postgresql
```

11. Prueba de escritura distribuida

Genera un archivo de prueba de 1 MiB dentro del NameNode:

```
podman exec bigdata-hdfs-namenode `
    bash -c "dd if=/dev/urandom of=/tmp/prueba-hdfs.bin bs=1M count=1 status=none"
```

Calcula su checksum local:

```
podman exec bigdata-hdfs-namenode `
    sha256sum /tmp/prueba-hdfs.bin
```

Copia el archivo a HDFS:

```
podman exec bigdata-hdfs-namenode `
    hdfs dfs -put -f `
```

```
/tmp/prueba-hdfs.bin `
/data/raw/prueba-hdfs.bin
```

Comprueba:

```
podman exec bigdata-hdfs-namenode `
hdfs dfs -ls -h /data/raw
```

12. Verificar la replicación física

Ejecuta:

```
podman exec bigdata-hdfs-namenode `
hdfs fsck `
/data/raw/prueba-hdfs.bin `
-files `
-blocks `
-locations
```

Busca una salida similar a:

```
1 blocks
repl=2
```

Y ubicaciones correspondientes a ambos DataNodes:

```
datanode1
datanode2
```

También debe terminar con:

Status: HEALTHY

Esta prueba demuestra que:

1. El NameNode registró el archivo.
 2. El archivo fue dividido en un bloque.
 3. El bloque posee dos réplicas.
 4. Cada réplica está almacenada en un DataNode diferente.
-

13. Validar integridad de lectura

Calcula el checksum desde HDFS:

```
podman exec bigdata-hdfs-namenode `
    bash -c "hdfs dfs -cat /data/raw/prueba-hdfs.bin | sha256sum"
```

El hash debe coincidir con el obtenido previamente mediante:

```
podman exec bigdata-hdfs-namenode `
    sha256sum /tmp/prueba-hdfs.bin
```

14. Prueba académica de tolerancia a caída

Esta prueba no destruye los datos.

Detener DataNode 2

```
podman stop bigdata-hdfs-datanode2
```

Comprueba el estado:

```
podman exec bigdata-hdfs-namenode `
    hdfs dfsadmin -report |
    Select-String `
        "Live datanodes|Dead datanodes"
```

Después intenta leer el archivo:

```
podman exec bigdata-hdfs-namenode `
    bash -c "hdfs dfs -cat /data/raw/prueba-hdfs.bin | sha256sum"
```

La lectura debería continuar funcionando porque existe una réplica en el DataNode restante.

Recuperar el nodo

```
podman start bigdata-hdfs-datanode2
```

Espera aproximadamente 20 segundos:

```
Start-Sleep -Seconds 20
```

Valida:

```
podman exec bigdata-hdfs-namenode `
    hdfs dfsadmin -report |
    Select-String "Live datanodes"
```

Resultado esperado:

```
Live datanodes (2)
```

Finalmente:

```
podman exec bigdata-hdfs-namenode `
    hdfs fsck /data/raw -files -blocks
```

15. Prueba de persistencia

Detén el clúster sin eliminar los volúmenes:

```
podman compose down
```

Levántalo nuevamente:

```
podman compose up -d
```

Espera:

```
Start-Sleep -Seconds 30
```

Comprueba que el archivo continúa disponible:

```
podman exec bigdata-hdfs-namenode `
    hdfs dfs -ls -h /data/raw
```

Y valida su estado:

```
podman exec bigdata-hdfs-namenode `
    hdfs fsck /data/raw/prueba-hdfs.bin `
    -files `
    -blocks `
    -locations
```

Esto confirma que los metadatos y los bloques permanecieron en los volúmenes administrados por Podman.

Estado esperado al finalizar

FUENTE

- └─ PostgreSQL
 - └─ 5 tablas
 - └─ 20.610 registros

ALMACENAMIENTO

- └─ HDFS
 - └─ NameNode
 - └─ DataNode 1

- └─ DataNode 2
- └─ factor de replicación 2
- └─ almacenamiento persistente
- └─ prueba de caída validada

Riesgos y limitaciones actuales

- El NameNode es un punto único de falla.
- No existe NameNode de alta disponibilidad.
- No hay Kerberos.
- Los permisos HDFS están desactivados.
- No existe cifrado en tránsito ni en reposo.
- Los DataNodes pertenecen a la misma máquina física.
- La tolerancia demostrada corresponde a caída lógica de contenedores, no a caída física independiente.

Esta configuración es apropiada para aprendizaje y pruebas funcionales, no para producción.

Cuando confirmes que `hdfs dfsadmin -report` muestra **dos DataNodes vivos** y `hdfs fsck` termina en **HEALTHY**, continuaremos con la primera **ingesta batch PostgreSQL → HDFS**, generando archivos CSV, manifiestos de ejecución y validación de conteos.

You said:

estoy en la etapa 7, revisando el log del contenedor

ChatGPT said:

El log identifica claramente el problema:

```
Directory /tmp/hadoop-root/dfs/name is in an inconsistent state:
storage directory does not exist or is not accessible
```

El NameNode intenta usar el volumen montado en:

```
/tmp/hadoop-root/dfs/name
```

pero el directorio no fue creado o el usuario interno de la imagen no tiene permisos para inicializarlo. Además, en el primer arranque el almacenamiento del NameNode debe formatearse antes de iniciar el servicio.

La imagen oficial es una *convenience image* orientada a pruebas, y Apache recomienda utilizar su configuración Compose asociada como referencia. Nuestra definición simplificada necesita encargarse explícitamente de crear y formatear los volúmenes. 🌸

hadoop.apache.org 📄 hub.docker.com

Corrección recomendada

No elimines el volumen de PostgreSQL. Eliminaremos solamente los tres volúmenes HDFS, que todavía no contienen información útil.

1. Detener los servicios HDFS

Desde la raíz del proyecto:

```
Set-Location "C:\rutinas-local\gen-ai-app\bigdata-poc"
```

```
podman compose stop namenode datanode1 datanode2
podman compose rm -f namenode datanode1 datanode2
```

Verifica los volúmenes existentes:

```
podman volume ls |
  Select-String "bigdata_hdfs"
```

Elimina exclusivamente los volúmenes HDFS:

```
podman volume rm bigdata_hdfs_namenode_data
podman volume rm bigdata_hdfs_datanode1_data
podman volume rm bigdata_hdfs_datanode2_data
```

Esto no elimina:

```
bigdata_postgres_data
```

2. Corregir los servicios HDFS en compose.yml

Reemplaza las definiciones de namenode, datanode1 y datanode2 por las siguientes.

NameNode

```
namenode:
  image: docker.io/apache/hadoop:3.5.0
  container_name: bigdata-hdfs-namenode
  hostname: namenode
  restart: unless-stopped

  user: "0:0"

  command:
    - bash
    - -c
```



```

- |
  set -e

  DIRECTORIO_NAMENODE="/tmp/hadoop-root/dfs/name"

  echo "[INFO] Preparando directorio del NameNode..."
  mkdir -p "${DIRECTORIO_NAMENODE}"

  if [ ! -f "${DIRECTORIO_NAMENODE}/current/VERSION" ]; then
    echo "[INFO] Formateando NameNode por primera vez..."
    hdfs namenode -format \
      -force \
      -nonInteractive \
      -clusterId bigdata-poc-cluster
  else
    echo "[INFO] El NameNode ya está formateado."
  fi

  echo "[INFO] Iniciando NameNode..."
  exec hdfs namenode

env_file:
- ./config/hadoop/hadoop.env

ports:
- "9870:9870"

volumes:
- hdfs_namenode_data:/tmp/hadoop-root/dfs/name

networks:
- bigdata_net

```

DataNode 1

```

datanode1:
  image: docker.io/apache/hadoop:3.5.0
  container_name: bigdata-hdfs-datanode1
  hostname: datanode1
  restart: unless-stopped

  user: "0:0"

  command:
  - bash
  - -c
  - |
    set -e

```

```

DIRECTORIO_DATANODE="/tmp/hadoop-root/dfs/data"

echo "[INFO] Preparando directorio del DataNode 1..."
mkdir -p "${DIRECTORIO_DATANODE}"

echo "[INFO] Esperando disponibilidad del NameNode..."
until bash -c "echo > /dev/tcp/namenode/8020" 2>/dev/null; do
    sleep 3
done

echo "[INFO] Iniciando DataNode 1..."
exec hdfs datanode

env_file:
- ./config/hadoop/hadoop.env

depends_on:
- namenode

ports:
- "9864:9864"

volumes:
- hdfs_datanode1_data:/tmp/hadoop-root/dfs/data

networks:
- bigdata_net

```

DataNode 2

```

datanode2:
  image: docker.io/apache/hadoop:3.5.0
  container_name: bigdata-hdfs-datanode2
  hostname: datanode2
  restart: unless-stopped

  user: "0:0"

  command:
    - bash
    - -c
    - |
      set -e

    DIRECTORIO_DATANODE="/tmp/hadoop-root/dfs/data"

    echo "[INFO] Preparando directorio del DataNode 2..."

```

```

mkdir -p "${DIRECTORIO_DATANODE}"

echo "[INFO] Esperando disponibilidad del NameNode..."
until bash -c "echo > /dev/tcp/namenode/8020" 2>/dev/null; do
    sleep 3
done

echo "[INFO] Iniciando DataNode 2..."
exec hdfs datanode

env_file:
- ./config/hadoop/hadoop.env

depends_on:
- namenode

ports:
- "9865:9864"

volumes:
- hdfs_datanode2_data:/tmp/hadoop-root/dfs/data

networks:
- bigdata_net

```

Qué corrige este cambio

La modificación realiza cuatro acciones:

1. Ejecuta la inicialización con permisos suficientes dentro del contenedor.
2. Crea explícitamente el directorio montado.
3. Formatea el NameNode solo cuando el volumen está vacío.
4. Evita reformatearlo durante reinicios posteriores.

La condición:

```
if [ ! -f "${DIRECTORIO_NAMENODE}/current/VERSION" ]
```

hace que el arranque sea idempotente. El formateo ocurre una sola vez.

3. Verificar `hadoop.env`

Confirma que tenga exactamente estas propiedades:

```
CORE-SITE.XML_fs.defaultFS=hdfs://namenode:8020
```

```
HDFS-SITE.XML_dfs.namenode.rpc-address=namenode:8020
```

```
HDFS-SITE.XML_dfs.replication=2
```

```
HDFS-SITE.XML_dfs.namenode.name.dir=file:///tmp/hadoop-root/dfs/name
```

```
HDFS-SITE.XML_dfs.datanode.data.dir=file:///tmp/hadoop-root/dfs/data
```

```
HDFS-SITE.XML_dfs.permissions.enabled=false
```

```
HDFS-SITE.XML_dfs.namenode.datanode.registration.ip-hostname-check=false
```

No dejes espacios al comienzo de las líneas ni comillas alrededor de los valores.

4. Validar Compose

Ejecuta:

```
podman compose config
```

Si quieres guardar una copia para diagnóstico:

```
podman compose config |  
  Set-Content `  
    -Path ".\logs\compose-resuelto.yml" `  
    -Encoding UTF8
```

Comprueba específicamente:

```
podman compose config |  
  Select-String `  
    "dfs.namenode.name.dir|dfs.datanode.data.dir|dfs.replication"
```

5. Levantar primero el NameNode

Para aislar el diagnóstico, no levantes todavía todo el clúster:

```
podman compose up -d namenode
```

Observa el estado:

```
podman ps -a `  
  --filter "name=bigdata-hdfs-namenode" `  
  --format "table {{.Names}}\t{{.Status}}"
```

Revisa el log:

```
podman logs --tail 150 bigdata-hdfs-namenode
```

Deberían aparecer mensajes semejantes a:

```
[INFO] Preparando directorio del NameNode...  
[INFO] Formateando NameNode por primera vez...  
Storage directory ... has been successfully formatted  
[INFO] Iniciando NameNode...
```

Y posteriormente:

```
NameNode RPC up at: namenode/...  
Web-server up at: 0.0.0.0:9870
```

Valida que permanezca ejecutándose:

```
podman ps ` `   
  --filter "name=bigdata-hdfs-namenode"
```

6. Revisar que el almacenamiento se inicializó

Ejecuta:

```
podman exec bigdata-hdfs-namenode ` `   
  bash -c "find /tmp/hadoop-root/dfs/name -maxdepth 3 -type f -print"
```

Deberían aparecer archivos como:

```
/tmp/hadoop-root/dfs/name/current/VERSION  
/tmp/hadoop-root/dfs/name/current/fsimage_...  
/tmp/hadoop-root/dfs/name/current/seen_txid
```

También revisa permisos:

```
podman exec bigdata-hdfs-namenode ` `   
  bash -c "id; ls -ld /tmp/hadoop-root/dfs/name; ls -la /tmp/hadoop-  
root/dfs/name/current"
```

7. Levantar los DataNodes

Solo cuando el NameNode permanezca operativo:

```
podman compose up -d datanode1 datanode2
```

Espera:

```
Start-Sleep -Seconds 20
```

Revisa los contenedores:

```
podman ps -a `
  --filter "name=bigdata-hdfs" `
  --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Logs del primer DataNode:

```
podman logs --tail 100 bigdata-hdfs-datanode1
```

Logs del segundo:

```
podman logs --tail 100 bigdata-hdfs-datanode2
```

8. Validación definitiva

Ejecuta:

```
podman exec bigdata-hdfs-namenode `
  hdfs dfsadmin -report
```

Debes encontrar:

Live datanodes (2)

Luego:

```
podman exec bigdata-hdfs-namenode `
  hdfs dfsadmin -report |
  Select-String `
    "Live datanodes|Dead datanodes"
```

Resultado esperado:

Live datanodes (2)

Dead datanodes (0)

Finalmente abre:

<http://localhost:9870>

Nota operacional

Usar:

```
user: "0:0"
```

es una concesión para este laboratorio local y permite inicializar los volúmenes administrados por Podman. No es el diseño deseable para producción. En una versión endurecida convendría crear una imagen derivada que:

- prepare los directorios durante la construcción;
- asigne propiedad al usuario Hadoop;
- ejecute los servicios como usuario no privilegiado;
- incorpore health checks;
- restrinja capacidades del contenedor.

En esta PoC priorizamos reproducibilidad y resolución controlada del problema antes de aplicar hardening.

You said:

estoy en el punto 4 de la revision:

ChatGPT said:

El error tiene **dos causas distintas**:

1. Docker Compose intenta interpretar `${DIRECTORIO_NAMENODE}` y `${DIRECTORIO_DATANODE}` como variables del archivo `.env`, antes de que el comando llegue al contenedor.
2. `services must be a mapping` indica que la estructura o indentación del `compose.yml` quedó inválida.

Para evitar ambos problemas, reemplaza el archivo completo por una versión corregida que usa rutas literales y no variables internas de Bash.

1. Reemplazar `compose.yml`

Desde:

```
C:\rutinas-local\gen-ai-app\bigdata-poc
```

abre:

```
notepad .\compose.yml
```

Reemplaza todo el contenido por:

```
name: bigdata-poc
```

```
services:
```

```

# =====
# FUENTE: POSTGRESQL
# =====
postgresql:
  image: docker.io/library/postgres:17
  container_name: bigdata-postgresql
  hostname: postgresql
  restart: unless-stopped

  environment:
    POSTGRES_DB: ${POSTGRES_DB}
    POSTGRES_USER: ${POSTGRES_USER}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}

  ports:
    - "${POSTGRES_PORT}:5432"

  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./database/postgresql/init:/docker-entrypoint-initdb.d:ro

  healthcheck:
    test:
      - CMD-SHELL
      - pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}
    interval: 10s
    timeout: 5s
    retries: 10
    start_period: 20s

  networks:
    - bigdata_net

# =====
# HDFS: NAMENODE
# =====
namenode:
  image: docker.io/apache/hadoop:3.5.0
  container_name: bigdata-hdfs-namenode
  hostname: namenode
  restart: unless-stopped

  user: "0:0"

  command:
    - bash
    - -c

```



```

- |
  set -e

  echo "[INFO] Preparando directorio del NameNode..."
  mkdir -p /tmp/hadoop-root/dfs/name

  if [ ! -f /tmp/hadoop-root/dfs/name/current/VERSION ]; then
    echo "[INFO] Formateando NameNode por primera vez..."
    hdfs namenode -format \
      -force \
      -nonInteractive \
      -clusterId bigdata-poc-cluster
  else
    echo "[INFO] El NameNode ya está formateado."
  fi

  echo "[INFO] Iniciando NameNode..."
  exec hdfs namenode

env_file:
- ./config/hadoop/hadoop.env

ports:
- "9870:9870"

volumes:
- hdfs_namenode_data:/tmp/hadoop-root/dfs/name

networks:
- bigdata_net

# =====
# HDFS: DATANODE 1
# =====
datanode1:
  image: docker.io/apache/hadoop:3.5.0
  container_name: bigdata-hdfs-datanode1
  hostname: datanode1
  restart: unless-stopped

  user: "0:0"

  command:
    - bash
    - -c
    - |
      set -e

```

```

    echo "[INFO] Preparando directorio del DataNode 1..."
    mkdir -p /tmp/hadoop-root/dfs/data

    echo "[INFO] Esperando disponibilidad del NameNode..."
    until bash -c "echo > /dev/tcp/namenode/8020" 2>/dev/null; do
        sleep 3
    done

    echo "[INFO] Iniciando DataNode 1..."
    exec hdfs datanode

env_file:
- ./config/hadoop/hadoop.env

depends_on:
- namenode

ports:
- "9864:9864"

volumes:
- hdfs_datanode1_data:/tmp/hadoop-root/dfs/data

networks:
- bigdata_net

# =====
# HDFS: DATANODE 2
# =====
datanode2:
    image: docker.io/apache/hadoop:3.5.0
    container_name: bigdata-hdfs-datanode2
    hostname: datanode2
    restart: unless-stopped

    user: "0:0"

    command:
    - bash
    - -c
    - |
        set -e

        echo "[INFO] Preparando directorio del DataNode 2..."
        mkdir -p /tmp/hadoop-root/dfs/data

        echo "[INFO] Esperando disponibilidad del NameNode..."
        until bash -c "echo > /dev/tcp/namenode/8020" 2>/dev/null; do

```

```
    sleep 3
done

echo "[INFO] Iniciando DataNode 2..."
exec hdfs datanode
```

```
env_file:
- ./config/hadoop/hadoop.env
```

```
depends_on:
- namenode
```

```
ports:
- "9865:9864"
```

```
volumes:
- hdfs_datanode2_data:/tmp/hadoop-root/dfs/data
```

```
networks:
- bigdata_net
```

```
volumes:
  postgres_data:
    name: bigdata_postgres_data

  hdfs_namenode_data:
    name: bigdata_hdfs_namenode_data

  hdfs_datanode1_data:
    name: bigdata_hdfs_datanode1_data

  hdfs_datanode2_data:
    name: bigdata_hdfs_datanode2_data
```

```
networks:
  bigdata_net:
    name: bigdata_net
```

2. Verificar .env

El archivo .env debe contener, al menos:

```
POSTGRES_DB=ventas
POSTGRES_USER=bigdata
POSTGRES_PASSWORD=BigData_PoC_2026
POSTGRES_PORT=5432
```

Comprueba:

```
Get-Content .\.env
```

3. Verificar hadoop.env

```
Get-Content .\config\hadoop\hadoop.env
```

Contenido esperado:

```
CORE-SITE.XML_fs.defaultFS=hdfs://namenode:8020
HDFS-SITE.XML_dfs.namenode.rpc-address=namenode:8020
HDFS-SITE.XML_dfs.replication=2
HDFS-SITE.XML_dfs.namenode.name.dir=file:///tmp/hadoop-root/dfs/name
HDFS-SITE.XML_dfs.datanode.data.dir=file:///tmp/hadoop-root/dfs/data
HDFS-SITE.XML_dfs.permissions.enabled=false
HDFS-SITE.XML_dfs.namenode.datanode.registration.ip-hostname-check=false
```

4. Validar nuevamente Compose

```
podman compose config
```

Ahora no deberían aparecer estos mensajes:

```
The "DIRECTORIO_NAMENODE" variable is not set
The "DIRECTORIO_DATANODE" variable is not set
services must be a mapping
```

Puedes comprobar los servicios reconocidos:

```
podman compose config --services
```

Resultado esperado:

```
postgresql
namenode
datanode1
datanode2
```

5. Limpiar solo los componentes HDFS fallidos

```
podman compose stop namenode datanode1 datanode2
podman compose rm -f namenode datanode1 datanode2
```

Elimina los volúmenes HDFS inicializados incorrectamente:

```
podman volume rm bigdata_hdfs_namenode_data
podman volume rm bigdata_hdfs_datanode1_data
```

```
podman volume rm bigdata_hdfs_datanode2_data
```

No elimines:

```
bigdata_postgres_data
```

Si algún volumen indica que no existe, puedes continuar.

6. Levantar únicamente el NameNode

```
podman compose up -d namenode
```

Revisa el estado:

```
podman ps -a `
  --filter "name=bigdata-hdfs-namenode" `
  --format "table {{.Names}}\t{{.Status}}"
```

Revisa el log:

```
podman logs --tail 150 bigdata-hdfs-namenode
```

Debes observar:

```
[INFO] Preparando directorio del NameNode...
[INFO] Formateando NameNode por primera vez...
...
[INFO] Iniciando NameNode...
```

Y el contenedor debe permanecer en estado Up.

7. Validar que se creó el filesystem del NameNode

```
podman exec bigdata-hdfs-namenode `
  bash -c "ls -la /tmp/hadoop-root/dfs/name/current"
```

Deberían aparecer archivos como:

```
VERSION
seen_txid
fsimage_...
```

8. Levantar los DataNodes

```
podman compose up -d datanode1 datanode2
```

Espera unos segundos:

Start-Sleep -Seconds 20

Comprueba:

```
podman ps -a `
  --filter "name=bigdata-hdfs" `
  --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Finalmente:

```
podman exec bigdata-hdfs-namenode `
  hdfs dfsadmin -report |
  Select-String "Live datanodes|Dead datanodes"
```

Resultado esperado:

```
Live datanodes (2)
Dead datanodes (0)
```

La causa principal fue la expansión anticipada de variables por Compose. Usar rutas literales elimina esa ambigüedad y deja el comando independiente del proveedor Compose utilizado en Windows.

You said:

Listo: hdfs dfsadmin -report muestra dos DataNodes vivos y hdfs fsck termina en HEALTHY

continuemos con la primera ingesta batch PostgreSQL → HDFS, generando archivos CSV, manifiestos de ejecución y validación de conteos.

ChatGPT said:

Perfecto. Implementaremos la primera ruta funcional:

```
PostgreSQL
|
| SELECT ordenado por clave primaria
▼
Contenedor Python
├─ genera CSV
├─ calcula SHA-256
├─ valida cantidad de registros
├─ carga mediante WebHDFS
└─ genera manifiesto JSON
    |
    ▼
HDFS
/data/raw/batch/postgresql/...
/data/manifests/batch/postgresql/...
```

La carga será reproducible, trazable y no sobrescribirá ejecuciones anteriores.

Fase 3: ingesta batch PostgreSQL → HDFS

1. Habilitar explícitamente WebHDFS

Edita:

```
notepad .\config\hadoop\hadoop.env
```

Agrega al final:

```
HDFS-SITE.XML_dfs.webhdfs.enabled=true
```

El archivo completo debe mantener, entre otras, estas propiedades:

```
CORE-SITE.XML_fs.defaultFS=hdfs://namenode:8020
```

```
HDFS-SITE.XML_dfs.namenode.rpc-address=namenode:8020
```

```
HDFS-SITE.XML_dfs.replication=2
```

```
HDFS-SITE.XML_dfs.namenode.name.dir=file:///tmp/hadoop-root/dfs/name
```

```
HDFS-SITE.XML_dfs.datanode.data.dir=file:///tmp/hadoop-root/dfs/data
```

```
HDFS-SITE.XML_dfs.permissions.enabled=false
```

```
HDFS-SITE.XML_dfs.namenode.datanode.registration.ip-hostname-check=false
```

```
HDFS-SITE.XML_dfs.webhdfs.enabled=true
```

Recrea solo los contenedores HDFS para aplicar la nueva variable:

```
podman compose up -d --force-recreate namenode datanode1 datanode2
```

```
Start-Sleep -Seconds 20
```

Valida:

```
podman exec bigdata-hdfs-namenode `
    hdfs dfsadmin -report |
    Select-String "Live datanodes"
```

Resultado esperado:

```
Live datanodes (2)
```

2. Crear los archivos del componente batch

Ubícate en el proyecto:

```
Set-Location "C:\rutinas-local\gen-ai-app\bigdata-poc"
```

Crea esta estructura:

```
New-Item `
  -ItemType Directory `
  -Path ".\ingestion\batch\src" `
  -Force |
  Out-Null
```

```
New-Item `
  -ItemType Directory `
  -Path ".\data\staging" `
  -Force |
  Out-Null
```

```
New-Item `
  -ItemType Directory `
  -Path ".\manifests" `
  -Force |
  Out-Null
```

La estructura relevante será:

```
ingestion
├── batch
│   ├── Containerfile
│   ├── requirements.txt
│   └── src
│       └── main.py
```

```
data
└── staging
```

```
manifests
```

3. Crear requirements.txt

Crea:

```
notepad .\ingestion\batch\requirements.txt
```

Contenido:

```
psycopg[binary]==3.2.9
requests==2.32.4
```

4. Crear el Containerfile

Crea:

```
notepad .\ingestion\batch\Containerfile
```

Contenido:

```
FROM docker.io/library/python:3.13-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt /app/requirements.txt

RUN pip install \
    --no-cache-dir \
    --disable-pip-version-check \
    -r /app/requirements.txt

COPY src /app/src

RUN mkdir -p \
    /app/data/staging \
    /app/manifests

ENTRYPOINT ["python", "/app/src/main.py"]
```

5. Crear el programa de ingesta

Crea:

```
notepad .\ingestion\batch\src\main.py
```

Contenido:

```
from __future__ import annotations

import csv
import hashlib
import json
import logging
import os
import sys
import time
```

```

from dataclasses import asdict, dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Any
from urllib.parse import quote

import psycopg
import requests

TABLAS: dict[str, str] = {
    "sucursales": "sucursal_id",
    "clientes": "cliente_id",
    "productos": "producto_id",
    "ventas": "venta_id",
    "detalle_ventas": "detalle_id",
}

DIRECTORIO_STAGING = Path("/app/data/staging")
DIRECTORIO_MANIFIESTOS = Path("/app/manifests")

ZONA_HDFS_DATOS = "/data/raw/batch/postgresql"
ZONA_HDFS_MANIFIESTOS = "/data/manifests/batch/postgresql"

TIEMPO_ESPERA_SERVICIOS_SEGUNDOS = 120
INTERVALO_REINTENTO_SEGUNDOS = 5

@dataclass
class ResultadoTabla:
    tabla: str
    clave_ordenamiento: str
    filas_origen: int
    filas_exportadas: int
    columnas: list[str]
    archivo_local: str
    archivo_hdfs: str
    tamano_bytes: int
    sha256: str
    estado: str
    duracion_segundos: float
    error: str | None = None

def configurar_logging() -> None:
    logging.basicConfig(
        level=logging.INFO,
        format=(
            "%(asctime)s | %(levelname)s | "

```

```

        "%(name)s | %(message)s"
    ),
    datefmt="%Y-%m-%dT%H:%M:%S",
)

```

```

def obtener_variable(nombre: str) -> str:
    valor = os.getenv(nombre)

    if not valor:
        raise RuntimeError(
            f"La variable de entorno obligatoria '{nombre}' no está definida."
        )

    return valor

```

```

def construir_configuracion() -> dict[str, Any]:
    return {
        "host_postgresql": obtener_variable("POSTGRES_HOST"),
        "puerto_postgresql": int(
            obtener_variable("POSTGRES_PORT_INTERNO")
        ),
        "base_postgresql": obtener_variable("POSTGRES_DB"),
        "usuario_postgresql": obtener_variable("POSTGRES_USER"),
        "contrasena_postgresql": obtener_variable(
            "POSTGRES_PASSWORD"
        ),
        "webhdfs_url": obtener_variable("WEBHDFS_URL").rstrip("/"),
        "usuario_hdfs": os.getenv("HDFS_USER", "root"),
    }

```

```

def esperar_postgresql(configuracion: dict[str, Any]) -> None:
    logger = logging.getLogger("esperar_postgresql")
    limite = time.monotonic() + TIEMPO_ESPERA_SERVICIOS_SEGUNDOS

    while time.monotonic() < limite:
        try:
            with psycopg.connect(
                host=configuracion["host_postgresql"],
                port=configuracion["puerto_postgresql"],
                dbname=configuracion["base_postgresql"],
                user=configuracion["usuario_postgresql"],
                password=configuracion["contrasena_postgresql"],
                connect_timeout=5,
            ) as conexion:
                with conexion.cursor() as cursor:
                    cursor.execute("SELECT 1")

```

```

        cursor.fetchone()

        logger.info("PostgreSQL está disponible.")
        return

    except psycopg.Error as error:
        logger.warning(
            "PostgreSQL aún no está disponible: %s",
            error,
        )
        time.sleep(INTERVALO_REINTENTO_SEGUNDOS)

    raise TimeoutError(
        "PostgreSQL no estuvo disponible dentro del tiempo esperado."
    )

def construir_url_webhdfs(
    configuracion: dict[str, Any],
    ruta_hdfs: str,
    operacion: str,
    **parametros: str,
) -> str:
    ruta_codificada = quote(ruta_hdfs, safe="/")

    argumentos = {
        "op": operacion,
        "user.name": configuracion["usuario_hdfs"],
        **parametros,
    }

    consulta = "&".join(
        f"{quote(str(clave))}={quote(str(valor))}"
        for clave, valor in argumentos.items()
    )

    return (
        f"{configuracion['webhdfs_url']}"
        f"/webhdfs/v1{ruta_codificada}?{consulta}"
    )

def crear_directorio_hdfs(
    configuracion: dict[str, Any],
    ruta_hdfs: str,
) -> None:
    url = construir_url_webhdfs(
        configuracion,
        ruta_hdfs,
    )

```

```

        "MKDIRS",
    )

    respuesta = requests.put(url, timeout=30)
    respuesta.raise_for_status()

    resultado = respuesta.json()

    if not resultado.get("boolean", False):
        raise RuntimeError(
            f"No fue posible crear el directorio HDFS: {ruta_hdfs}"
        )

def subir_archivo_hdfs(
    configuracion: dict[str, Any],
    ruta_local: Path,
    ruta_hdfs: str,
) -> None:
    logger = logging.getLogger("subir_archivo_hdfs")

    url_inicial = construir_url_webhdfs(
        configuracion,
        ruta_hdfs,
        "CREATE",
        overwrite="false",
    )

    respuesta_inicial = requests.put(
        url_inicial,
        allow_redirects=False,
        timeout=30,
    )

    if respuesta_inicial.status_code != 307:
        raise RuntimeError(
            "WebHDFS no devolvió la redirección esperada. "
            f"HTTP={respuesta_inicial.status_code}, "
            f"respuesta={respuesta_inicial.text[:500]}"
        )

    url_datanode = respuesta_inicial.headers.get("Location")

    if not url_datanode:
        raise RuntimeError(
            "WebHDFS no devolvió la ubicación del DataNode."
        )

    logger.info(

```

```

        "Subiendo %s hacia %s",
        ruta_local.name,
        ruta_hdfs,
    )

    with ruta_local.open("rb") as archivo:
        respuesta_carga = requests.put(
            url_datanode,
            data=archivo,
            timeout=300,
        )

    respuesta_carga.raise_for_status()

def calcular_sha256(ruta_archivo: Path) -> str:
    hash_archivo = hashlib.sha256()

    with ruta_archivo.open("rb") as archivo:
        for bloque in iter(lambda: archivo.read(1024 * 1024), b''):
            hash_archivo.update(bloque)

    return hash_archivo.hexdigest()

def obtener_conexion(
    configuracion: dict[str, Any],
) -> psycopg.Connection[Any]:
    return psycopg.connect(
        host=configuracion["host_postgresql"],
        port=configuracion["puerto_postgresql"],
        dbname=configuracion["base_postgresql"],
        user=configuracion["usuario_postgresql"],
        password=configuracion["contrasena_postgresql"],
    )

def contar_filas(
    conexion: psycopg.Connection[Any],
    tabla: str,
) -> int:
    with conexion.cursor() as cursor:
        cursor.execute(
            psycopg.sql.SQL("SELECT COUNT(*) FROM {}").format(
                psycopg.sql.Identifier(tabla)
            )
        )

    resultado = cursor.fetchone()

```

```

if resultado is None:
    raise RuntimeError(
        f"No se pudo obtener el conteo de la tabla {tabla}."
    )

return int(resultado[0])

def exportar_tabla_csv(
    conexion: psycopg.Connection[Any],
    tabla: str,
    clave_ordenamiento: str,
    ruta_csv: Path,
) -> tuple[int, list[str]]:
    filas_exportadas = 0

    consulta = psycopg.sql.SQL(
        "SELECT * FROM {} ORDER BY {}".format(
            psycopg.sql.Identifier(tabla),
            psycopg.sql.Identifier(clave_ordenamiento),
        )

    with conexion.cursor(name=f"cursor_batch_{tabla}") as cursor:
        cursor.itsize = 1000
        cursor.execute(consulta)

        columnas = [
            descripcion.name
            for descripcion in cursor.description or
        ]

        with ruta_csv.open(
            "w",
            encoding="utf-8",
            newline="",
        ) as archivo_csv:
            escritor = csv.writer(
                archivo_csv,
                delimiter=",",
                quotechar='"',
                quoting=csv.QUOTE_MINIMAL,
                lineterminator="\n",
            )

            escritor.writerow(columnas)

            for fila in cursor:

```

```

        escritor.writerow(fila)
        filas_exportadas += 1

    return filas_exportadas, columnas

def procesar_tabla(
    conexion: psycpg.Connection[Any],
    configuracion: dict[str, Any],
    tabla: str,
    clave_ordenamiento: str,
    id_ejecucion: str,
    fecha_carga: str,
    directorio_ejecucion: Path,
) -> ResultadoTabla:
    logger = logging.getLogger(f"tabla.{tabla}")
    inicio = time.monotonic()

    ruta_local = directorio_ejecucion / f"{tabla}.csv"

    ruta_directorio_hdfs = (
        f"{ZONA_HDFS_DATOS}/{tabla}"
        f"/fecha_carga={fecha_carga}"
        f"/id_ejecucion={id_ejecucion}"
    )

    ruta_archivo_hdfs = (
        f"{ruta_directorio_hdfs}/{tabla}.csv"
    )

    try:
        logger.info("Contando filas de origen.")
        filas_origen = contar_filas(conexion, tabla)

        logger.info("Exportando tabla a CSV.")
        filas_exportadas, columnas = exportar_tabla_csv(
            conexion=conexion,
            tabla=tabla,
            clave_ordenamiento=clave_ordenamiento,
            ruta_csv=ruta_local,
        )

        if filas_origen != filas_exportadas:
            raise RuntimeError(
                "El conteo de filas no coincide: "
                f"origen={filas_origen}, "
                f"exportadas={filas_exportadas}"
            )

```



```

sha256 = calcular_sha256(ruta_local)
tamano_bytes = ruta_local.stat().st_size

crear_directorio_hdfs(
    configuracion,
    ruta_directorio_hdfs,
)

subir_archivo_hdfs(
    configuracion,
    ruta_local,
    ruta_archivo_hdfs,
)

duracion = round(time.monotonic() - inicio, 3)

logger.info(
    "Tabla completada: filas=%d, bytes=%d, sha256=%s",
    filas_exportadas,
    tamano_bytes,
    sha256,
)

return ResultadoTabla(
    tabla=tabla,
    clave_ordenamiento=clave_ordenamiento,
    filas_origen=filas_origen,
    filas_exportadas=filas_exportadas,
    columnas=columnas,
    archivo_local=str(ruta_local),
    archivo_hdfs=ruta_archivo_hdfs,
    tamano_bytes=tamano_bytes,
    sha256=sha256,
    estado="OK",
    duracion_segundos=duracion,
)

except Exception as error:
    duracion = round(time.monotonic() - inicio, 3)

    logger.exception(
        "Falló la ingesta de la tabla %s.",
        tabla,
    )

    return ResultadoTabla(
        tabla=tabla,
        clave_ordenamiento=clave_ordenamiento,
        filas_origen=0,

```

```

        filas_exportadas=0,
        columnas=,
        archivo_local=str(ruta_local),
        archivo_hdfs=ruta_archivo_hdfs,
        tamano_bytes=0,
        sha256="",
        estado="ERROR",
        duracion_segundos=duracion,
        error=str(error),
    )

def guardar_manifiesto(
    configuracion: dict[str, Any],
    manifiesto: dict[str, Any],
    id_ejecucion: str,
    fecha_carga: str,
) -> Path:
    ruta_local = (
        DIRECTORIO_MANIFIESTOS
        / f"manifiesto_batch_postgresql_{id_ejecucion}.json"
    )

    with ruta_local.open(
        "w",
        encoding="utf-8",
    ) as archivo:
        json.dump(
            manifiesto,
            archivo,
            ensure_ascii=False,
            indent=2,
        )

    ruta_directorio_hdfs = (
        f"{ZONA_HDFS_MANIFIESTOS}"
        f"/fecha_carga={fecha_carga}"
    )

    ruta_hdfs = (
        f"{ruta_directorio_hdfs}/"
        f"manifiesto_{id_ejecucion}.json"
    )

    crear_directorio_hdfs(
        configuracion,
        ruta_directorio_hdfs,
    )

```

```

subir_archivo_hdfs(
    configuracion,
    ruta_local,
    ruta_hdfs,
)

return ruta_local

def main() -> int:
    configurar_logging()
    logger = logging.getLogger("ingesta_batch")

    fecha_inicio = datetime.now(timezone.utc)
    id_ejecucion = fecha_inicio.strftime("%Y%m%dT%H%M%SZ")
    fecha_carga = fecha_inicio.strftime("%Y-%m-%d")

    configuracion = construir_configuracion()

    directorio_ejecucion = (
        DIRECTORIO_STAGING / id_ejecucion
    )
    directorio_ejecucion.mkdir(
        parents=True,
        exist_ok=False,
    )

    logger.info(
        "Inicio de ingesta batch. id_ejecucion=%s",
        id_ejecucion,
    )

    resultados: list[ResultadoTabla] =

    try:
        esperar_postgresql(configuracion)

        with obtener_conexion(configuracion) as conexion:
            conexion.autocommit = False

            for tabla, clave in TABLAS.items():
                resultado = procesar_tabla(
                    conexion=conexion,
                    configuracion=configuracion,
                    tabla=tabla,
                    clave_ordenamiento=clave,
                    id_ejecucion=id_ejecucion,
                    fecha_carga=fecha_carga,
                    directorio_ejecucion=directorio_ejecucion,

```

```

        )
        resultados.append(resultado)

fecha_fin = datetime.now(timezone.utc)

tablas_error = [
    resultado.tabla
    for resultado in resultados
    if resultado.estado != "OK"
]

estado_general = (
    "OK"
    if not tablas_error
    else "ERROR"
)

manifiesto = {
    "version_manifiesto": "1.0",
    "tipo_ingesta": "batch",
    "motor_origen": "postgresql",
    "id_ejecucion": id_ejecucion,
    "fecha_inicio_utc": fecha_inicio.isoformat(),
    "fecha_fin_utc": fecha_fin.isoformat(),
    "estado": estado_general,
    "tablas_con_error": tablas_error,
    "cantidad_tablas": len(resultados),
    "filas_totales_origen": sum(
        resultado.filas_origen
        for resultado in resultados
    ),
    "filas_totales_exportadas": sum(
        resultado.filas_exportadas
        for resultado in resultados
    ),
    "resultados": [
        asdict(resultado)
        for resultado in resultados
    ],
}

ruta_manifiesto = guardar_manifiesto(
    configuracion=configuracion,
    manifiesto=manifiesto,
    id_ejecucion=id_ejecucion,
    fecha_carga=fecha_carga,
)

logger.info(

```

```

        "Manifiesto generado: %s",
        ruta_manifiesto,
    )

    logger.info(
        "Fin de ingesta batch. estado=%s, "
        "filas_exportadas=%d",
        estado_general,
        manifiesto["filas_totales_exportadas"],
    )

    return 0 if estado_general == "OK" else 1

except Exception:
    logger.exception(
        "La ejecución batch finalizó con un error no controlado."
    )
    return 2

if __name__ == "__main__":
    sys.exit(main())

```

Qué realiza el programa

Para cada tabla:

1. Obtiene el conteo en PostgreSQL.
 2. Ejecuta un `SELECT * ORDER BY clave_primaria`.
 3. Genera un CSV UTF-8 con encabezado.
 4. Cuenta las filas exportadas.
 5. Compara origen contra CSV.
 6. Calcula el hash SHA-256.
 7. Crea la ruta en HDFS.
 8. Sube el archivo a HDFS.
 9. Registra el resultado en un manifiesto JSON.
-

6. Agregar el servicio batch a `compose.yml`

Agrega este servicio dentro de `services:`, al mismo nivel que `postgresql`, `namenode` y `datanode1`.

No lo agregues dentro de otro servicio.

```
ingesta-batch:
  build:
    context: ./ingestion/batch
    dockerfile: Containerfile

  image: localhost/bigdata-poc-ingesta-batch:0.1
  container_name: bigdata-ingesta-batch
  hostname: ingesta-batch

  restart: "no"

  environment:
    POSTGRES_HOST: postgresql
    POSTGRES_PORT_INTERNO: "5432"
    POSTGRES_DB: ${POSTGRES_DB}
    POSTGRES_USER: ${POSTGRES_USER}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}

    WEBHDFS_URL: http://namenode:9870
    HDFS_USER: root

  volumes:
    - ./data/staging:/app/data/staging
    - ./manifests:/app/manifests

  depends_on:
    postgresql:
      condition: service_healthy

    namenode:
      condition: service_started

    datanode1:
      condition: service_started

    datanode2:
      condition: service_started

  networks:
    - bigdata_net
```

Consideración sobre depends_on

El proveedor Compose puede interpretar condition de manera distinta según su implementación. El programa incluye reintentos para PostgreSQL, pero no para WebHDFS. Por ello ejecutaremos la ingesta solo después de validar manualmente los servicios.

7. Validar el archivo Compose

```
podman compose config
```

Comprueba los servicios:

```
podman compose config --services
```

Resultado esperado:

```
postgresql
namenode
datanode1
datanode2
ingesta-batch
```

Si aparece `services must be a mapping`, revisa que `ingesta-batch:` tenga exactamente dos espacios de indentación bajo `services:`.

8. Construir la imagen de ingesta

```
podman compose build ingesta-batch
```

Comprueba:

```
podman images |
  Where-Object Repository -Match "bigdata-poc-ingesta-batch"
```

9. Verificar previamente los servicios

```
podman ps `
  --format "table {{.Names}}\t{{.Status}}"
```

Deben estar activos:

```
bigdata-postgresql
bigdata-hdfs-namenode
bigdata-hdfs-datanode1
bigdata-hdfs-datanode2
```

Valida PostgreSQL:

```
podman inspect bigdata-postgresql `
  --format "{{.State.Health.Status}}"
```

Resultado esperado:

healthy

Valida HDFS:

```
podman exec bigdata-hdfs-namenode `
    hdfs dfsadmin -report |
    Select-String "Live datanodes"
```

Resultado esperado:

Live datanodes (2)

10. Prueba de conectividad WebHDFS

Antes de ejecutar toda la carga:

```
podman run --rm `
    --network bigdata_net `
    docker.io/curlimages/curl:8.14.1 `
    -fsS `
    "http://namenode:9870/webhdfs/v1/?op=LISTSTATUS&user.name=root"
```

Debe devolver JSON y no un error HTTP.

Una salida abreviada podría comenzar así:

```
{"FileStatuses":{"FileStatus":[...]}}
```

11. Ejecutar la ingesta batch

Ejecuta como proceso de una sola ejecución:

```
podman compose run --rm ingesta-batch
```

Durante la ejecución deberías ver mensajes como:

```
Inicio de ingesta batch
PostgreSQL está disponible
Exportando tabla a CSV
Subiendo clientes.csv
Tabla completada
...
Fin de ingesta batch. estado=OK
```

El contenedor temporal será eliminado por `--rm`, pero los CSV y manifiestos permanecerán en los directorios montados.

12. Revisar los archivos locales

```
Get-ChildItem `
    -Path ".\data\staging" `
    -Recurse `
    -File |
    Select-Object `
        FullName,
        Length,
        LastWriteTime
```

Debes encontrar cinco CSV:

```
sucursales.csv
clientes.csv
productos.csv
ventas.csv
detalle_ventas.csv
```

Revisa los manifiestos:

```
Get-ChildItem `
    -Path ".\manifests" `
    -Filter "*.json" |
    Sort-Object LastWriteTime -Descending
```

Para mostrar el último:

```
$UltimoManifiesto = Get-ChildItem `
    -Path ".\manifests" `
    -Filter "*.json" |
    Sort-Object LastWriteTime -Descending |
    Select-Object -First 1
```

```
Get-Content $UltimoManifiesto.FullName -Raw |
    ConvertFrom-Json |
    Format-List
```

13. Verificar los archivos en HDFS

```
podman exec bigdata-hdfs-namenode `
    hdfs dfs -ls -R /data/raw/batch/postgresql
```

La estructura esperada será:

```
/data/raw/batch/postgresql/clientes
/data/raw/batch/postgresql/clientes/fecha_carga=2026-07-25
/data/raw/batch/postgresql/clientes/fecha_carga=2026-07-25/id_ejecucion=...
/data/raw/batch/postgresql/clientes/.../clientes.csv
```

Revisa los manifiestos:

```
podman exec bigdata-hdfs-namenode `
hdfs dfs -ls -R /data/manifests/batch/postgresql
```

14. Validar conteos origen contra CSV en HDFS

Primero muestra los conteos de PostgreSQL:

```
podman exec bigdata-postgresql `
psql `
-U bigdata `
-d ventas `
-c @"
SELECT 'sucursales' AS tabla, COUNT(*) AS filas FROM sucursales
UNION ALL
SELECT 'clientes', COUNT(*) FROM clientes
UNION ALL
SELECT 'productos', COUNT(*) FROM productos
UNION ALL
SELECT 'ventas', COUNT(*) FROM ventas
UNION ALL
SELECT 'detalle_ventas', COUNT(*) FROM detalle_ventas
ORDER BY tabla;
"@
```

Valores esperados:

Tabla	Registros
clientes	500
detalle_ventas	15.000
productos	100
sucursales	10
ventas	5.000

Como cada CSV posee una línea de encabezado, la cantidad de datos es:

`líneas CSV - 1`

Obtén el CSV más reciente de clientes y cuenta sus líneas:

```
podman exec bigdata-hdfs-namenode `
  bash -c '
    ARCHIVO=$(hdfs dfs -ls -R /data/raw/batch/postgresql/clientes |
      awk "/clientes.csv$/ {print \$8}" |
      tail -1)

    echo "Archivo: ${ARCHIVO}"

    LINEAS=$(hdfs dfs -cat "${ARCHIVO}" | wc -l)

    echo "Líneas totales: ${LINEAS}"
    echo "Registros sin encabezado: $((LINEAS - 1))"
  `
```

Para clientes, debe devolver:

Registros sin encabezado: 500

15. Validación automática de las cinco tablas

Ejecuta:

```
podman exec bigdata-hdfs-namenode `
  bash -c '
    for TABLA in sucursales clientes productos ventas detalle_ventas
    do
      ARCHIVO=$(hdfs dfs -ls -R \
        "/data/raw/batch/postgresql/${TABLA}" 2>/dev/null |
        awk -v tabla="${TABLA}.csv" "\$8 ~ tabla\"\\$\" {print \$8}" |
        tail -1)

      if [ -z "${ARCHIVO}" ]; then
        echo "${TABLA}|ERROR|archivo no encontrado"
        continue
      fi

      LINEAS=$(hdfs dfs -cat "${ARCHIVO}" | wc -l)
      REGISTROS=$((LINEAS - 1))

      echo "${TABLA}|OK|${REGISTROS}|${ARCHIVO}"
    done
  `
```

```
done
```

Salida esperada:

```
sucursales|0K|10|...
clientes|0K|500|...
productos|0K|100|...
ventas|0K|5000|...
detalle_ventas|0K|15000|...
```

16. Verificar distribución y salud de los archivos

```
podman exec bigdata-hdfs-namenode `
    hdfs fsck `
    /data/raw/batch/postgresql `
    -files `
    -blocks `
    -locations
```

Debe finalizar con:

Status: HEALTHY

Busca además:

```
repl=2
```

Los archivos CSV serán pequeños y, normalmente, cada uno utilizará un solo bloque con dos réplicas.

17. Revisar una muestra de datos

Encabezado y primeras filas de ventas:

```
podman exec bigdata-hdfs-namenode `
    bash -c '
        ARCHIVO=$(hdfs dfs -ls -R \
            /data/raw/batch/postgresql/ventas |
            awk "/ventas.csv$/ {print \$8}" |
            tail -1)

        hdfs dfs -cat "${ARCHIVO}" |
        head -n 6
    '
```

Salida conceptual:

```
venta_id,cliente_id,sucursal_id,fecha_venta,estado,total
1,1,1,2025-01-01 08:00:00,PAGADA,...
2,2,2,2025-01-01 08:10:00,PAGADA,...
```

18. Comportamiento ante una segunda ejecución

Al ejecutar nuevamente:

```
podman compose run --rm ingesta-batch
```

se creará otra partición:

```
fecha_carga=2026-07-25/
├─ id_ejecucion=20260725T064500Z/
└─ id_ejecucion=20260725T071000Z/
```

Esto evita sobrescribir la ejecución anterior y permite:

- comparar cargas;
- auditar resultados;
- reproducir pruebas;
- conservar trazabilidad temporal.

Resultado esperado de esta fase

FUENTE

```
└─ PostgreSQL
   └─ 5 tablas / 20.610 registros
```

INGESTA BATCH

```
└─ Python
   ├── extracción ordenada
   ├── CSV UTF-8
   ├── conteo origen/exportado
   ├── SHA-256
   └─ manifiesto JSON
```

ALMACENAMIENTO

```
└─ HDFS
   ├── partición por fecha
   ├── partición por ejecución
   ├── replicación 2
   └─ estado HEALTHY
```

La siguiente etapa natural será incorporar **MySQL con el mismo modelo lógico** y reutilizar el componente batch mediante una abstracción mínima por tipo de motor, sin duplicar toda la lógica.